NANYANG TECHNOLOGICAL UNIVERSITY

# NEXT-GEN GENERATIVE SEARCH ENGINE FOR CODE BASE

Timothy Lee Hongyi

College of Computing and Data Science

2026

# NANYANG TECHNOLOGICAL UNIVERSITY

**SC4079**

**NEXT-GEN GENERATIVE SEARCH ENGINE FOR CODE BASE**

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Computing in Data Science and Artificial Intelligence
of the Nanyang Technological University

by

Timothy Lee Hongyi

College of Computing and Data Science
2026

# Abstract

This report presents the design, implementation and evaluation of CodeOrient, an autonomous Artificial Intelligence (AI) search tool, designed to accelerate developer onboarding. This application leverages Natural Language Processing (NLP), code graph visualisation and vector-based retrieval to help new developers understand unfamiliar codebases and reduce the time to first commit.

CodeOrient integrates semantic code search using embedding models, structural analysis through dependency graphs and generative user interfaces (UI) to provide context-aware feature cards. Through the use of Retrieval-Augmented Generation (RAG) with source grounding, the application eliminates hallucination, commonly seen in AI code assistants. Preliminary testing suggests that by externalising the mental model of a codebase through a unified graph-and-card interface, the system significantly reduces the cognitive load associated with onboarding and system discovery in large-scale repositories.

# Acknowledgments

I would like to express my deepest gratitude to my supervisor, Professor Tan Chee Wei, for his unwavering guidance and patience throughout the duration of this Final Year Project. His expertise and insightful suggestions were key in shaping the direction of this project. I am very honoured to have had the opportunity to work under his mentorship.

# Contents

**Abstract** i

**Acknowledgments** ii

**Contents** iii

**List of Tables** viii

**List of Figures** ix

**List of Listings** x

**List of Algorithms** xi

**1 Introduction** 1

1.1 Motivation and Problem Statement ..... 1

1.2 Research Objectives and Goals ..... 2

1.3 Key Contributions ..... 2

1.4 Report Structure ..... 3

**2 Literature Review** 5

2.1 Software Complexity Metrics and Code Quality Measurements ..... 6

2.1.1 Cyclomatic Complexity as Foundation ..... 6

2.1.2 Modern Complexity in Distributed Systems ..... 7

2.2 Code Search and Natural Language Query Processing ..... 7

2.2.1 Semantic Code Search Through Embeddings ..... 7

2.2.2 Structural Code Search with Domain-Specific Languages ..... 8

2.2.3 Retrieval using Code Embedding Models ..... 8

2.3 AI Hallucination and Grounded Code Comprehension ..... 9
2.3.1 The Hallucination Problem in LLMs ..... 9
2.3.2 Citation-Grounded Code Comprehension ..... 10
2.3.3 Retrieval-Augmented Generation ..... 10
2.4 Code Visualisation and Dependency Analysis ..... 10
2.4.1 Static Analysis and Dependency Graphs ..... 10
2.4.2 Interactive Visualisation Tools ..... 11
2.5 Generative User Interface (UI) ..... 11
2.5.1 Adaptive and Dynamic Interfaces ..... 11
2.6 Research Gaps and Motivation ..... 11
2.6.1 Bridging Semantic and Structural Code Search ..... 12
2.6.2 The "Black Box" of Generative UI in Software Engineering ..... 12
2.7 Conclusion ..... 13

**3 System Design and Architecture** **14**
3.1 System Overview ..... 14
3.1.1 Core Components ..... 14
3.1.2 Data Flow Architecture ..... 15
3.1.3 Code Indexing Phase ..... 16
3.1.4 Query Response Phase ..... 16
3.1.5 Serverless Service Architecture ..... 17
3.2 Frontend Architecture ..... 18
3.2.1 Technology Stack ..... 18
3.2.2 User Interface Design ..... 19
3.2.3 Interactive Graph Visualisation with React Flow ..... 20
3.2.4 Generative UI Card Components ..... 21
3.3 Backend Architecture ..... 21
3.3.1 API Design ..... 21
3.3.2 Code Search Engine ..... 22
3.3.3 LLM Integration and RAG Pipeline ..... 22
3.4 Data Pipeline ..... 23

3.4.1 Repository Ingestion ..... 23
3.4.2 Chunking Strategy ..... 23
3.4.3 Metadata Extraction ..... 24
3.4.4 Storage of Code Chunks ..... 24
3.5 Technical Stack Summary ..... 25

**4 Implementation Details** **26**
4.1 Development Methodology ..... 26
4.1.1 Iterative Development Process ..... 26
4.1.2 Version Control and Branching Strategy ..... 27
4.2 Core Implementation Components ..... 28
4.2.1 Search Module ..... 28
4.2.2 Code Graph Analysis ..... 30
4.2.3 Generative UI System ..... 32
4.2.4 Large Language Model Integration ..... 34
4.2.5 RAG Pipeline Implementation ..... 36
4.3 Database Schema ..... 38
4.4 UI/UX Enhancements ..... 41
4.4.1 Account Dashboard ..... 41
4.4.2 Sharing of Conversations ..... 41
4.4.3 Setting Preferences ..... 42
4.5 Challenges and Solutions ..... 43
4.5.1 Handling Large Codebases ..... 43
4.5.2 Real-Time Graph Updates ..... 44

**5 Evaluation Methodology** **45**
5.1 Evaluation Framework ..... 45
5.1.1 Research Questions ..... 45
5.1.2 Hypotheses ..... 46
5.2 Experimental Setup ..... 46
5.2.1 Dataset ..... 46
5.2.2 Baseline Comparisons ..... 46

5.3 Quantitative Metrics ..... 47
5.3.1 Recall@System ..... 47
5.3.2 Success@Turn ..... 47
5.3.3 Graph Recall ..... 47
5.3.4 Faithfulness (Hallucination Rate) ..... 48
5.3.5 Latency and Caching Efficiency ..... 48
5.3.6 Clicks to Destination (CTD) ..... 48

**6 Results and Discussions** **49**
6.1 Quantitative Results ..... 49
6.2 Comparative Analysis with Baselines ..... 49
6.2.1 Performance vs GitHub Search ..... 49
6.2.2 Performance vs Ablation Model ..... 50
6.3 Key Findings ..... 50
6.3.1 Impact of Graph Visualisation and Emergent Inference ..... 50
6.3.2 Effectiveness of Citation Grounding ..... 51
6.4 Case Studies ..... 51
6.4.1 Case Study 1: "How does the system resolve and inject dependencies?" ..... 51
6.4.2 Case Study 2: "How are the button visual variants defined?" ..... 52
6.4.3 Case Study 3: "How is dark mode theme switching implemented?" ..... 53
6.5 Implications for Developer Onboarding ..... 54
6.6 Limitations ..... 55
6.6.1 Technical Limitations ..... 55
6.6.2 Threats to Validity ..... 55

**7 Conclusion** **57**
7.1 Summary of Contributions ..... 57
7.2 Key Takeaways ..... 57
7.3 Future Work ..... 58

**A Detailed Results** **60**

A.1 Quantitative Results Table ..... 60

# List of Tables

3.1 *Summary of Technical Stack and Rationale* ..... 25

6.1 *Average Evaluation Metrics Across 30 Queries* ..... 49

A.1 *Complete Quantitative Results by Query* ..... 64

# List of Figures


2.1 *Control flow graph of a simple if-else statement.* ..... 6
2.2 *Semantic code search in a shared embedding space for retrieval [3].* ..... 8
2.3 *Exploiting LLM hallucinations through slopsquatting.* ..... 9
2.4 *Retrieval-Augmented Generation to reduce hallucinations [7].* ..... 10
2.5 *Generative UI for a Room Rug Visualiser [11].* ..... 12

3.1 *Data Flow Architecture of CodeOrient* ..... 15
3.2 *CodeOrient Chat Page Layout* ..... 19
3.3 *Graph Visualisation with Interactive Tooltips* ..... 20

4.1 *Prototype of Dynamic Card Generation for Weather App* ..... 27
4.2 *Git Branching Strategy for CodeOrient Development* ..... 28
4.3 *Example of Repository Card in Generative UI* ..... 33
4.4 *Example of Code Graph Card in Generative UI* ..... 34
4.5 *Example of Multistep Planning by Search Architect Persona* ..... 35
4.6 *Example of Gap Analysis by Gap Analyser Persona* ..... 35
4.7 *Example of sources retrieved from the RAG pipeline* ..... 37
4.8 *Example of inline citations in LLM response* ..... 38
4.9 *Example of Account Dashboard* ..... 41
4.10 *Example of Sharing Conversation Link* ..... 42
4.11 *Example of User Preferences Settings* ..... 43

6.1 *Graph visualisation for Case Study 2* ..... 53
6.2 *Graph visualisation for Case Study 3* ..... 54

# List of Listings

1 *TypeScript interfaces for CodeOrient graph entities.* . . . . . 32
2 *Example of Assembled Context with Source Metadata.* . . . . . 37
3 *Prisma Database Schema.* . . . . . 40

# List of Algorithms

1 *Language-Aware Recursive Code Splitting Algorithm* . . . . . 31

# Chapter 1

# Introduction

Software Engineering is undergoing a fundamental shift with the rise in Large Language Models (LLMs), Agentic Artificial Intelligence (AI), and generative user interfaces (GenUI). As the complexity of current software architectures (distributed systems) and codebases grows, the challenge of developers learning a new repository has become a significant bottleneck for engineering productivity. I propose CodeOrient, an autonomous AI-driven developer onboarding platform that leverages Retrieval-Augmented Generation (RAG) with dynamic graph visualisation. Deployed as an interactive onboarding assistant, CodeOrient combines the reasoning capabilities of LLMs with the structural insights of code graphs to transform how developers understand and navigate unfamiliar codebases.

## 1.1 Motivation and Problem Statement

The rapid advancement of AI-assisted coding tools, such as GitHub Copilot and Cursor, has prioritised code generation over code comprehension. As a new developer onboards, they often struggle with understanding the existing codebases due to their complex architectural nature. Three critical issues affecting effective onboarding are:

1. **LLM Hallucinations:** LLMs are trained on past data and often generate plausible but factually incorrect information. This is particularly problematic as the generated response has no knowledge of the specific codebase being queried.

2. **Limited Context Window:** Large codebases often exceed the input context window of LLMs, leading to incomplete or incorrect answers as the model cannot access all relevant information.

3. **Overloading of Information:** Most codebases are accompanied by static documentation, which fails to capture the dynamic relationships within the code. Without visual context, developers are often overwhelmed by the volume of code and struggle to identify relevant components in a large codebase.

## 1.2 Research Objectives and Goals

The primary goal of this research is to design and implement CodeOrient, an AI-driven developer onboarding tool that addresses the challenges of code comprehension in complex codebases. Addressing these challenges translate into the following specific objectives:

1. **Develop a Retrieval-Augmented Generation (RAG) Framework:** Implement a hybrid search mechanism that integrates vector-based semantic search with traditional keyword-based search to retrieve relevant code snippets and documentation.

2. **Implement Code Visualisation with Generative UI:** Utilise React Flow to dynamically render interactive graphs based on user queries.

3. **Minimise AI Hallucinations:** Ensure every response or claim made by the LLM is backed by code citations from the retrieved documents.

4. **Evaluate the Effectiveness of Dynamic Visualisation:** Evaluate how dynamic graph visualisations affect the comprehension ability of developers compared to traditional text-based documentation.

## 1.3 Key Contributions

CodeOrient introduces three key contributions that separate it from existing developer onboarding tools:

- **Dynamic UIs:** Instead of traditional text-based responses generated by LLMs, Generative UI is used to create dynamic visualisations from user queries. If a developer asks about "authentication flow," the UI creates a graph focused strictly on those related modules, rather than a cluttered, static diagram.
- **Citation-Grounded RAG Pipeline:** To reduce hallucination, CodeOrient integrates sources and inline citations mechanisms into the RAG framework, ensuring that all LLM responses can be verified against actual code segments.
- **Integration of Agentic LLMs:** CodeOrient utilises agentic LLMs equipped with tools. They can autonomously decide when to query the vector database, generate visualisations, or seek clarifications based on the conversation context.

The final outcome of this research is publicly accessible for demonstration at `https://codeorient.vercel.app`, and the source code is available at `https://github.com/timooo-thy/ai-search`.

## 1.4 Report Structure

This report is structured for the ideation to implementation journey of CodeOrient:

- **Chapter 2:** Literature Review explores software complexity metrics, semantic code search, and the emerging technologies of Generative UI.
- **Chapter 3:** System Design and Architecture details the technical stack of CodeOrient, which features the RAG pipeline, code graph generation, and the interactive chat interface.
- **Chapter 4:** Implementation Details discusses the development methodology, key algorithms, and integration challenges encountered during the build process.
- **Chapters 5 & 6:** Evaluation, Results and Discussion will analyse the tool's effectiveness to improve the onboarding experience through user studies and case studies on real-world codebases.
- **Chapter 7:** Conclusion summarises the contributions of this research and dis-

cusses potential future work to enhance CodeOrient's capabilities.

# Chapter 2

# Literature Review

New developers often struggle with understanding new and unfamiliar codebases, leading to prolonged onboarding times and reduced productivity. Various strategies have been proposed to address this challenge, such as improved documentation practices. However, these methods often fall short in providing comprehensive and efficient solutions for code comprehension. The rise in adoption of AI-assisted tools has introduced new challenges. One key challenge is that these tools hallucinate information that is often not grounded in actual source code. Furthermore, large codebases make it harder for LLMs to comprehend due to their limited context window.

This literature review examines five interrelated research domains critical to CodeOrient, an AI Search Tool for code comprehension:

1. Software Complexity Metrics and Code Quality Measurements
2. Semantic Code Search and Natural Language Query Processing
3. AI Hallucination and Grounded Code Comprehension
4. Code Visualisation and Dependency Analysis
5. Generative User Interfaces

The combination of these areas provides the theoretical foundation for building an AI application that reduces developer onboarding time while ensuring the generated

explanations are accurate and grounded in the actual codebase.

## 2.1 Software Complexity Metrics and Code Quality Measurements

### 2.1.1 Cyclomatic Complexity as Foundation

Thomas J.McCabe introduced cyclomatic complexity, a quantitative measure of program complexity based on control graph flow analysis [1]. His work established that cyclomatic complexity directly correlates with code maintainability and testing. The formula $M = E - N + 2P$, where $E$ represents edges, $N$ represents nodes, and $P$ represents the number of connected components in a control flow graph, and represents the complexity as the number of linearly independent paths through a program's source code [1]. As shown in *Figure 2.1*, a simple control flow graph of a function below yields a complexity of 2, where $P = 1$.

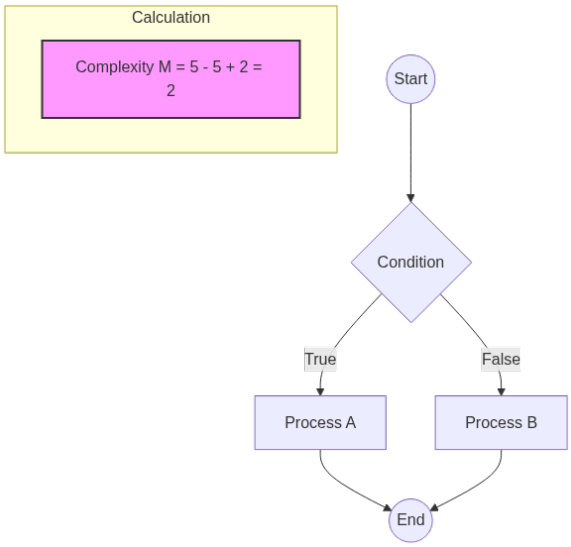



Figure 2.1: *Control flow graph of a simple if-else statement.*

McCabe's complexity measure allows developers to identify highly complex functions and recognise problematic code sections that require refactoring. When implementing the code graph visualisation feature in a code search application, cyclomatic complexity serves as one signal among many to highlight high-risk or critical code sections.

### 2.1.2 Modern Complexity in Distributed Systems

Kafura's recent reflection on McCabe's work in 2025 acknowledges that cyclomatic complexity has proven durable for the last 50 years. However, modern software architectures, such as distributed systems and microservices, require additional metrics beyond control flow analysis [2]. This observation showcases the potential of integrating graph visualisation into modern AI assistants to help developers understand both functional and architectural complexity.

## 2.2 Code Search and Natural Language Query Processing

### 2.2.1 Semantic Code Search Through Embeddings

Code search has evolved from simple keyword matching to more sophisticated semantic search techniques. Cambronero et al. explored the use of neural embeddings for semantic code search [3]. Their approach involves training models to transform both code snippets and natural language queries into a shared vector space [3]. Similar code snippets can be retrieved by calculating the cosine similarity between their embedding vectors. Cambronero et al. demonstrated that the use of neural embeddings could bridge the semantic gap between natural language queries and code snippets [3]. By transforming both code and queries into a shared vector space, code snippets relevant to the query can be retrieved by calculating the cosine similarity between their embedding vectors, given by the formula:

$$\text{cosine_similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\|\|\vec{b}\|}$$

where $\vec{a}$ and $\vec{b}$ are the embedding vectors of the query and code snippet, respectively.

*Figure 2.2* showcases how a query is transformed into an embedding vector which shares the same embedding space as code snippets. Although this technique does not replace traditional code search, it complements traditional methods that often miss

semantically relevant results.

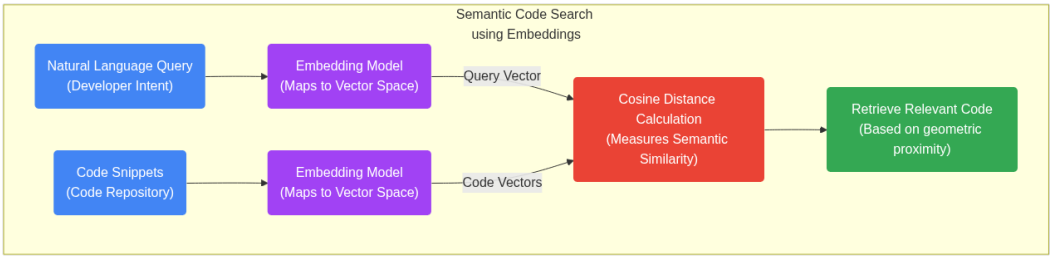



Figure 2.2: *Semantic code search in a shared embedding space for retrieval [3].*

### 2.2.2 Structural Code Search with Domain-Specific Languages

Recent research conducted in 2025 explored structural code search techniques using Domain-Specific Languages (DSLs). Limpanukorn et al. proposed translating natural language queries into DSL queries, which capture the structural relationships within a codebase, which goes beyond mere semantic similarity [4]. Their approach achieved a precision score of 55-70% and outperformed semantic search baselines by up to 57% on F1 scores [4].

This research presents a critical missing link in developer tools, such as tracing an "authentication flow", where the relationships between modules are more informative than the functions' names themselves. Their findings highlight the need for code graph visualisation to represent these structural relationships effectively.

### 2.2.3 Retrieval using Code Embedding Models

Qodo introduced specialised code embedding models (Qodo Embed-1) that achieved state-of-the-art performance (product score of 3.72/5) in Codebase Understanding Gartner® in 2025 [5]. Their approach bypasses the intermediate language description step. Instead, their models directly encode code semantics, resulting in the models being computationally efficient while maintaining high retrieval accuracy for code search tasks [5].

## 2.3 AI Hallucination and Grounded Code Comprehension

### 2.3.1 The Hallucination Problem in LLMs

With the increased adoption of AI-assisted code generation tools like GitHub Copilot and ChatGPT, the risk of hallucination has become a critical concern. Hallucination in code generation refers to the generation of code that appears correct but is actually non-functional. In 2025, Spracklen et al. conducted a comprehensive study on hallucination in code-generating LLMs [6] and found that package hallucinations are a systemic issue across state-of-the-art code-generating models. Their research included analysing over 576,000 code samples generated by 16 different LLMs. Their findings revealed that LLMs consistently hallucinate package names, and more worryingly, they regenerate the same false package name in 43% of repeated prompts [6].

In an agentic workflow where LLMs autonomously generate and execute code, hallucinations can be exploited by attackers via "slopsquatting", which is the practice of creating malicious packages with names similar to popular ones [6]. *Figure 2.3* showcases how an attacker can exploit hallucinations from LLMs. This research highlights the urgent need for citation-grounded code comprehension systems that can verify all claims against actual source code.

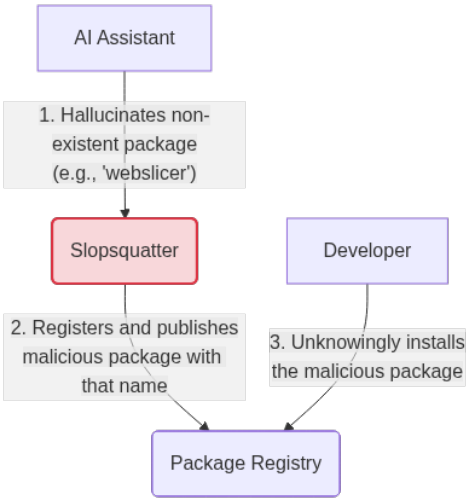



Figure 2.3: *Exploiting LLM hallucinations through slopsquatting.*

### 2.3.2 Citation-Grounded Code Comprehension

Arafat et al.'s recent work on citation-grounded code comprehension directly addresses the hallucination problem [7]. They conclude that code comprehension systems must ground all claims in verifiable source code citations. Their proposed hybrid retrieval system with Neo4j graph database to provide import relationships, achieved a 92% citation accuracy with zero hallucinations. Moreover, the graph component discovered richer cross-file relationships that purely text-based retrieval missed, 62% of architectural queries [7].

### 2.3.3 Retrieval-Augmented Generation

The broader principle emerging from hallucination research is Retrieval-Augmented Generation (RAG). As it is not possible to train new information into LLMs at scale, RAG provides a mechanism to ground LLM outputs with real-time data via a retriever [8]. The use of a retriever reduced hallucination rates across all categories from a baseline high of 21% to below 7.5% [8]. In the context of code comprehension, RAG refers to a retriever that fetches relevant code snippets, which are then passed to an LLM as a context to generate grounded explanations with source citations.

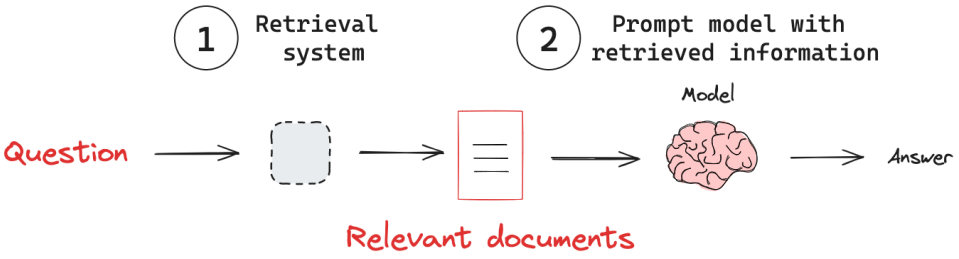



Figure 2.4: *Retrieval-Augmented Generation to reduce hallucinations [7].*

## 2.4 Code Visualisation and Dependency Analysis

### 2.4.1 Static Analysis and Dependency Graphs

To understand the real structure of a codebase, dependency graphs are essential. Using entities as nodes and relationships as edges, code graphs provide a visual representation

of how different components interact with each other [9]. The lack of visualisation makes it difficult for developers to grasp the complex relationship that are present in modern software architectures. This motivates CodeOrient's code graph visualisation feature to help developers trace data flows across multiple modules, functions, and files.

### 2.4.2 Interactive Visualisation Tools

Among many code visualisation libraries, React Flow stands out for its interactivity and ease of integration with React applications [10]. React Flow allows developers to create interactive visualisations of code relationships. Rather than traditional large static diagrams, developers can pan and hover over nodes to reveal additional information, which is crucial for understanding complex codebases.

## 2.5 Generative User Interface (UI)

### 2.5.1 Adaptive and Dynamic Interfaces

Generative UI is an emerging field of research by Google that focuses on using LLMs to generate dynamic UIs [11]. This extends the capability of text-based LLMs to generate UIs using task-driven data models [12]. Different queries will produce different UIs at runtime, allowing for highly personalised and adaptive interfaces. Their research shifts away from generating UI code snippets towards generating UI data models that are more aligned with user intent [12]. This approach directly motivates the use of Generative UI by rendering UI components instead of text-based responses in CodeOrient.

## 2.6 Research Gaps and Motivation

The reviewed literature highlights several advancements in software complexity metrics, semantic code search, hallucination mitigation, code visualisation, and Generative UI. However, there has been limited work in integrating these advancements into a cohesive system for developer onboarding. The following research gaps motivate the development of CodeOrient:

### 2.6.1 Bridging Semantic and Structural Code Search

Current tools typically favour either semantic search (finding code that looks right) or structural analysis (finding code that is connected). As noted by Limpanukorn et al. (2025), structural search outperforms semantic baselines, yet most AI tools like GitHub Copilot still relies primarily on text-based RAG. There is a lack of research into how Generative UI can bridge this gap by dynamically synthesising a visual graph that represents both the user's natural language intent and the codebase's physical architecture.

### 2.6.2 The "Black Box" of Generative UI in Software Engineering

Research by Leviathan et al. [11] and Cao et al. [12] establishes the framework for task-driven UIs. However, these studies focus on general tasks such as education or shopping, as shown in *Figure 2.5*. In the high-stakes domain of software engineering, it is unknown how a constantly changing, generative interface affects a developer's productivity. CodeOrient will serve as an experimental platform to explore whether Generative UI can effectively reduce cognitive load and accelerate code comprehension for developers.

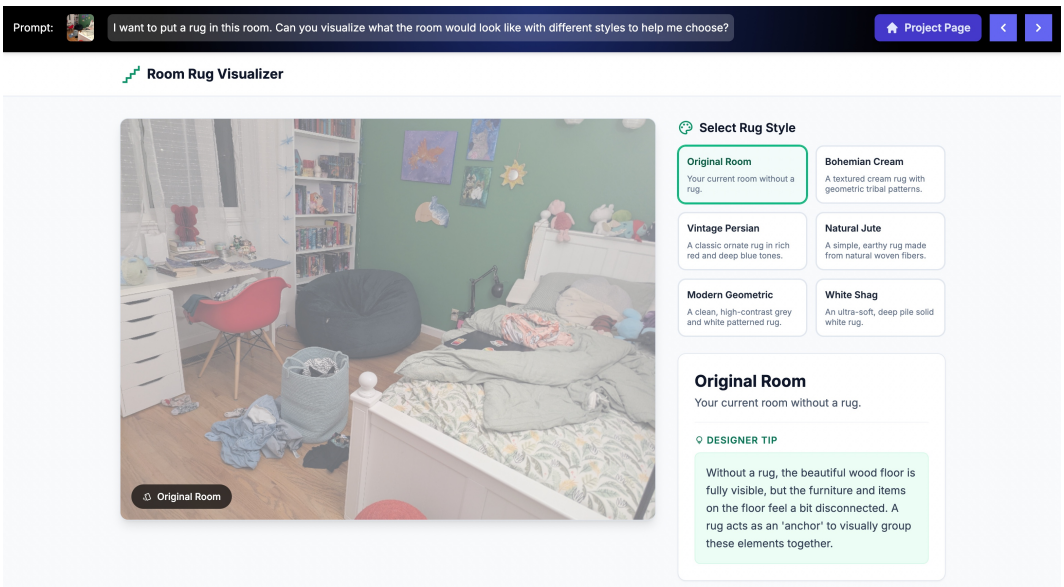



Figure 2.5: *Generative UI for a Room Rug Visualiser [11].*

## 2.7 Conclusion

To improve developer onboarding, it is essential to address code comprehension at different levels. By integrating insights from software complexity metrics [1], semantic code search [3], hallucination mitigation [7], and Generative UI [11], CodeOrient aims to accelerate developer onboarding while being reliable and grounded in actual source code.

# Chapter 3

# System Design and Architecture

## 3.1 System Overview

This chapter details the system architecture of CodeOrient. It will provide the high-level overview of the core components, data flow architecture, frontend and backend design, data pipeline, and technical stack summary.

### 3.1.1 Core Components

The design of CodeOrient is composed of four main components:

- **User Interface:** Built with Next.js and React Flow to provide an interactive experience between the user and AI assistant.
- **LLM Assistant:** Vercel AI SDK and OpenAI are utilised for natural language processing, tool calling, and response generation.
- **Code Search Engine:** A hybrid search engine that combines semantic search with keyword-based retrieval to find relevant code snippets which fallbacks to GitHub Search API when necessary.
- **Storage Solutions:** PostgreSQL is used for storing structured data, Upstash Redis for caching frequent queries, and Upstash Vector for storing vector embeddings of code chunks.

### 3.1.2 Data Flow Architecture

A multi-stage pipeline optimised for speed and quality of responses is built to handle user interactions. As shown in *Figure 3.1*, it illustrates the data flow, split into two main phases: *Code Indexing Phase* and *Query Response Phase*.

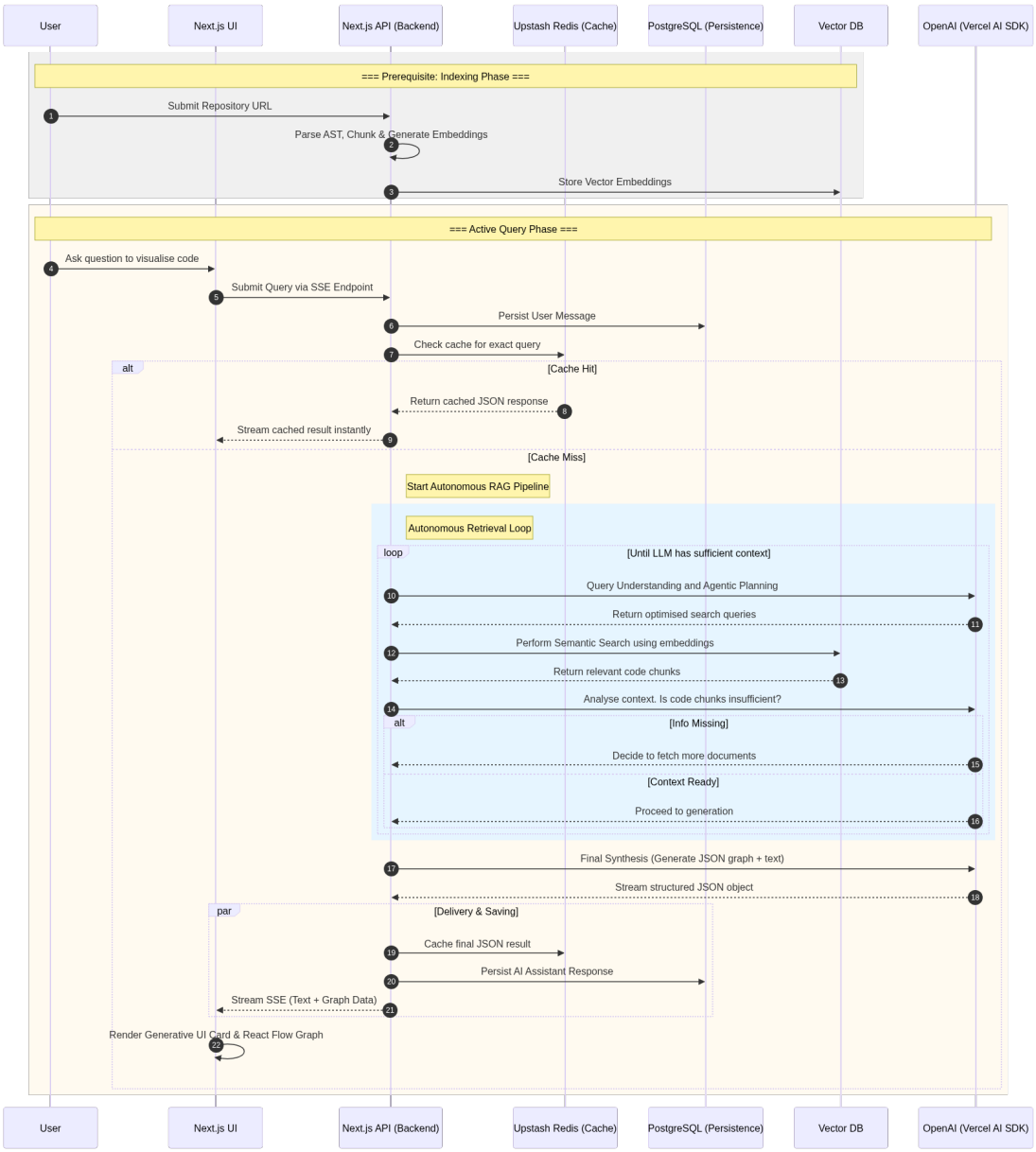



Figure 3.1: *Data Flow Architecture of CodeOrient*

### 3.1.3 Code Indexing Phase

For vector-based retrieval to work, code indexing for the required code repository is performed as a prerequisite step. This phase parses the source code files, chunks them, and generates vector embeddings to be stored in a vector database. To improve the recall, docstrings and comments are extracted as well. In cases where indexing is not performed, CodeOrient falls back to searching the codebase directly via GitHub Search API during query time.

### 3.1.4 Query Response Phase

This phase is triggered when a user submits a query. The steps involved are as follows:

1. **Query Submission:** The user submits a query via the frontend. The message is sent to the backend via API and saved to PostgreSQL to persist chat history.
2. **Cache Layer:** The backend will first check Upstash Redis for an identical query based on the current session ID. Two outcomes are possible:
    - **Cache Hit:** If a cached response exists, the stored JSON response is retrieved and streamed back to the frontend, cutting down latency by over 90%. Additionally, the use of a cache reduces overall LLM token consumption and cost.
    - **Cache Miss:** If no cached response is found, the system initiates the Retrieval-Augmented Generation (RAG) pipeline.
3. **Autonomous RAG Pipeline:** This pipeline represents the core reasoning engine of CodeOrient.
    - **Query Understanding and Planning:** The LLM first understands and rephrases the user's query. This step fixes potential errors, such as typos and ambiguous terms. If there is insufficient context, the LLM will prompt the user for more information instead. Next, the LLM generates a plan to decompose the query into sub-tasks, generating up to 3 sub-queries for retrieval.

- **Hybrid Retrieval and Ranking:** Each sub-query is converted into embeddings before retrieving the top-K relevant code chunks from the vector database. A ranking mechanism based on Distributed-Based Score Fusion (DBSF) is applied to combine results from both semantic and sparse search.
- **LLM Evaluation Loop:** This step utilises an autonomous and specialised LLM for context evaluation. If the context is deemed insufficient, the LLM will trigger additional searches in a loop until it gathers sufficient context to confidently answer the user query.

4. **Citation Grounded Response:** After gathering sufficient context, the LLM generates a structured JSON object that contains the required nodes, edges, code snippets (citations) and explanation required for visualisation. To mitigate hallucinations, the LLM is further prompted to perform inline citations for each code snippet used in the response.
5. **Message Persistence and Caching:** The final chat result is stored in PostgreSQL and also cached in Upstash Redis for future identical queries. Simultaneously, the generated response is streamed via Server-Sent-Events (SSE) back to the frontend for real-time rendering.
6. **Rendering Components:** Finally, the frontend parses the streamed JSON response to render as a Generative UI card, containing an interactive graph visualisation built with React Flow. Citation-backed explanations are streamed in after the graph is rendered.

### 3.1.5 Serverless Service Architecture

CodeOrient utilises Next.js API Route Handlers to create a modular, serverless backend. This separates logical concerns while maintaining the codebase as a monorepo for quicker development cycles. Additionally, when it is deployed in a serverless environment (Vercel), each Route Handler (e.g., `/api/search` vs `/api/index`) scales independently based on traffic. Even as a monorepo, the backend logic is divided into specialised services that operate independently:

- **Ingestion Service:** Handles the ingestion of code repositories from GitHub API asynchronously through background jobs, allowing the frontend to remain responsive.
- **Vector Orchestration Service:** Manages the communication with the vector database, which handles embedding generation and semantic search.
- **RAG Agent:** Specialised agent that has access to various tools such as Code Search, Graph Generation, etc. It maintains the state of the search by deciding if the retrieved content is sufficient to answer the query or if more code snippets are required.
- **Code Search Service:** Interfaces with GitHub Search API to discover relevant code snippets based on user queries.
- **Persistence and Cache Service:** Prisma, a dedicated Object Relational Mapping (ORM) service, is used to interact with PostgreSQL for persistence of chat history and a Redis client to interact with Upstash Redis for caching frequent queries.

## 3.2 Frontend Architecture

### 3.2.1 Technology Stack

- **Framework:** The framework used for development is Next.js App Router with Typescript. This framework allows the mix of Server Components (RSC) and Client Components to optimise for performance and developer experience. Server Components are used for static content that does not require interactivity, while Client Components are used for interactive elements such as the chat interface and graph visualisation. This separation allows for faster load times and improved performance by reducing the amount of JavaScript sent to the client.
- **Styling:** Tailwind CSS & Shadcn UI are utilised to design the user interface, providing a consistent and responsive design system.

### 3.2.2 User Interface Design

This section discusses the key design principles and layout of the chat page of CodeOrient application. *Figure 3.2* showcases the chat page layout, which features a split-pane design that prioritises the chat interface with the AI assistant and the source code.

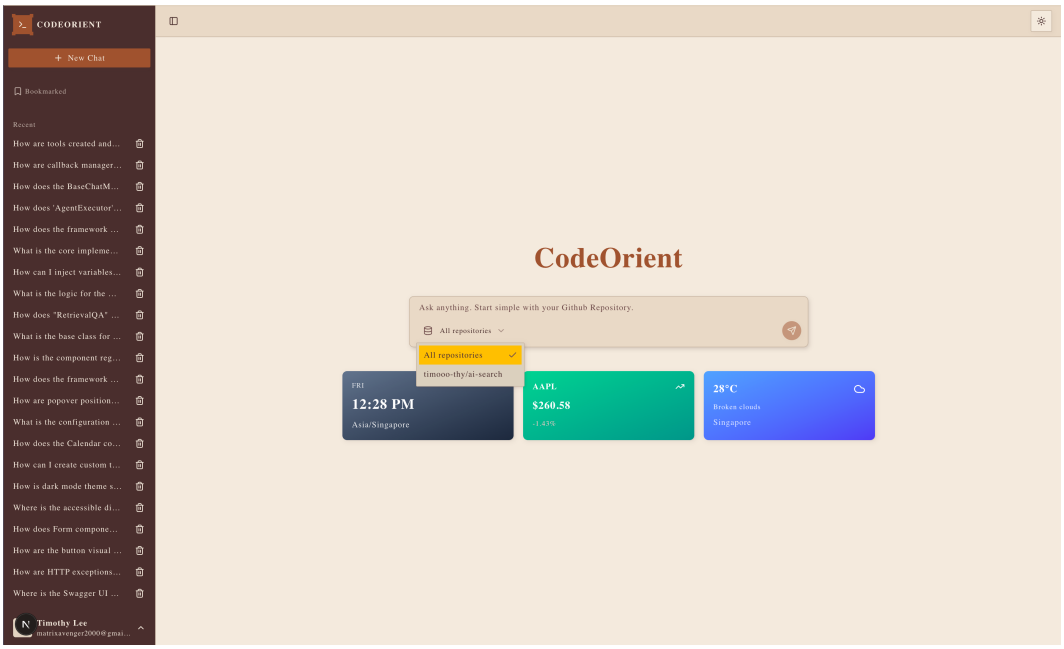



Figure 3.2: *CodeOrient Chat Page Layout*

The main components of the chat page layout include:

- **Persistent Sidebar:** A collapsible left sidebar is introduced to manage user context and provide quick access to conversation history in a chronological order. It also includes a user profile section at the bottom for account settings and logout.
- **Navigation Bar:** Sharing options, bookmarking and toggling of light/dark mode is included in the top navigation bar.
- **Main Chat Area:** This area is dedicated to the chat interface with the AI assistant.
- **Responsive Design:** The layout adapts to different screen sizes, ensuring usability across devices from mobile to desktop.

### 3.2.3 Interactive Graph Visualisation with React Flow

Static architecture diagrams are often high-level and extremely abstracted, making it difficult for users to zoom in on a specific module. To address this, CodeOrient integrates LLMs and React Flow to visualise a subset of the code repository as an interactive graph. The core components of a graph visualisation include:

- **Nodes as Entities:** Nodes represent high-level code entities such as files, functions, classes, and components. Each node contains metadata about the entity type, file path, code snippet and description.
- **Edges as Relationships:** Dependencies are represented as directed edges between nodes, illustrating how data and logic flow between different parts of the codebase, where common relationship types include imports, calls, extends, and uses. For example, Component A importing a Function B would be represented as an edge from Node A to Node B.
- **Interactive Features:** Additional interactive features, such as panning, zooming, or hovering over nodes, are implemented to improve code understanding and navigation within the codebase. In *Figure 3.3*, hovering over a node displays a tooltip containing more information about the node.

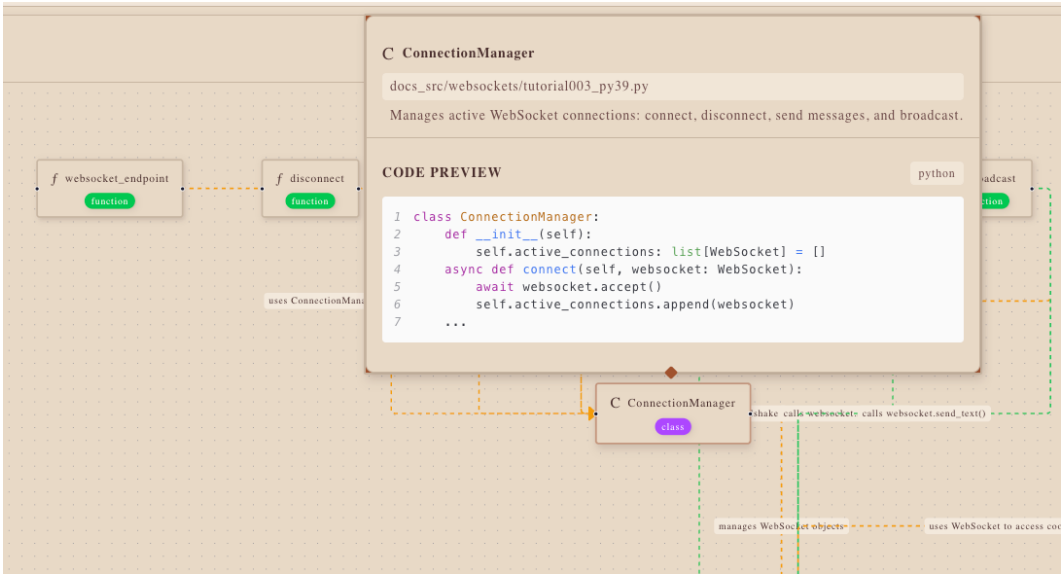



Figure 3.3: *Graph Visualisation with Interactive Tooltips*

### 3.2.4 Generative UI Card Components

Standard LLM responses are limited to text or markdown outputs. Often, this is insufficient to convey complex code structures or relationships. To address this, CodeOrient adopted a Generative UI architecture in which the LLM generates structured JSON objects that render as functional React components.

Based on the user's query, the LLM autonomously decides which component to generate. For example, if the user asks about their list of repositories, the LLM would generate a "Repository Card" component. To simulate a smoother user experience, the component data is streamed to the UI, allowing the card to populate incrementally and reducing perceived latency.

## 3.3 Backend Architecture

### 3.3.1 API Design

A RESTful API design pattern is adopted to achieve a clear separation of concerns among backend services. Each endpoint is implemented as a serverless function, allowing each service to scale independently based on demand. The API is divided into four primary services:

- **Authentication API:** Manages user login, registration, and session management.
- **Ingestion API:** Manages the lifecycle of ingesting and indexing code repositories from GitHub API.
- **Search & Retrieval API:** Interfaces with the vector and structured databases to perform semantic search, retrieval of code snippets and chat persistence.
- **Streaming Chat API:** Orchestrates the Vercel AI SDK to handle LLM generation via SSE to the client.

### 3.3.2 Code Search Engine

Traditional keyword-based or fuzzy search methods fail to capture the intent behind a user's query, especially in a large and complex code repository. To overcome this problem, CodeOrient utilises a semantic Code Search Engine for vector-based retrieval of relevant code snippets.

1. **GitHub Octokit:** The engine uses GitHub's Octokit library to fetch private repository contents from the user's account. This allows the system to access up-to-date codebases for indexing.

2. **Hybrid Retrieval Strategy:** The engine utilises `bge-large-en-v1.5`, an embedding model to perform semantic search in Upstash Vector. Furthermore, it conducts a sparse search to find documents based on keyword frequency and term importance.

3. **Filter Mechanism:** To prevent cross-project contamination, the vector database is partitioned by repository and user ID. To further refine search results, the engine supports metadata filtering, such as file type or entity type.

4. **Ranking Mechanism:** To balance between sparse and dense retrievals, retrieved code snippets are ranked using Distributed-Based Score Fusion (DBSF).

### 3.3.3 LLM Integration and RAG Pipeline

Creating an autonomous RAG pipeline enables the LLM to move beyond simple search-and-retrieval patterns. Instead, the LLM follows a more complex reasoning process:

1. **Query Rephrasing:** The LLM first rephrases the user's query into multiple optimised sub-queries to improve recall during retrieval.

2. **Autonomous Reasoning Loop:** The pipeline implements a loop where the LLM will evaluate the initial search results. If a certain context is missing, the LLM will autonomously generate secondary searches before finalising the response. For example, if the LLM finds a function call but is missing the definition, the agent identifies the missing piece and triggers another search specifically for that

function definition.

3. **Hallucination Mitigation:** To eliminate hallucinations, the LLM is prompted to only cite the relevant code snippets that are used to formulate the response. Each citation includes the file path, URL and code snippet for user verification. This ensures higher precision and trustworthiness of the generated content.

## 3.4 Data Pipeline

This section details the four-stage data pipeline that transforms code repositories into a structured knowledge base.

### 3.4.1 Repository Ingestion

The system uses an efficient streaming ingestion strategy to bypass GitHub API rate limits. The key optimisations are as follows:

1. The system fetches the entire repository as a compressed `.tar.gz` archive in a single request using GitHub's Octokit.
2. The system utilises LangChain to extract code chunks from the archive in memory, speeding up the ingestion process.
3. During ingestion, a whitelist containing indexable file types (e.g., `.ts`, `.py`, `.go`) is used to filter out non-essential files (e.g., DockerFile).

### 3.4.2 Chunking Strategy

Instead of indexing a single file as a single document, the engine splits each code file into smaller "logical" chunks, ensuring each chunk maintains coherence during retrieval.

1. The system uses `RecursiveCharacterTextSplitter` with its `.fromLanguage()` method to parse code files based on their programming language. The advantage of this approach is that it prioritises splitting at language-specific boundaries (e.g., functions, classes) rather than arbitrary character limits.

2. Each chunk is limited to 1,500 tokens with a chunk overlap of 200 tokens. This overlap is key to maintaining context across chunk boundaries, ensuring that related code segments are not lost during retrieval.

### 3.4.3 Metadata Extraction

To provide a richer context during retrieval, each code chunk includes additional metadata fields extracted during parsing.

1. The parser is utilised to scan for docstrings or comments preceding code entities. These high-level summaries provide semantic context during retrieval, improving the relevance of search results.

2. To provide accurate citations during response generation, each chunk is tagged with its `filePath`, `repoFullName`, specific `startLine` and `endLine`.

### 3.4.4 Storage of Code Chunks

The final stage involves storing the processed code chunks for low-latency retrieval during query time.

1. The system uses Prisma ORM and PostgresSQL to track the real-time progress of the indexing lifecycle of each repository to the user (e.g., `CLONING` → `PARSING` → `INDEXING` → `COMPLETED`). In the event of a network interruption, this serves as a checkpoint to resume indexing.

2. To optimise throughput, code chunks are converted into embeddings using the `bge-large-en-v1.5` model and upserted into Upstash Vector in batches of 100.

## 3.5 Technical Stack Summary

| Category | Technology | Engineering Justification |
|---|---|---|
| **Framework** | Next.js 16 | Enables a monorep setup for seamless integration of server and client components. |
| **AI Framework** | Vercel AI SDK | Enables real-time Generative UI rendering via SSE. |
| **Database & ORM** | PostgreSQL & Prisma | Manages the user, repository, and chat-session relational data while providing a type-safe query interface. |
| **Vector Store** | Upstash Vector | Provides a serverless vector database with metadata filtering to avoid cross-repository contamination. |
| **Caching** | Upstash Redis | Reduces LLM token consumption and decreases latency by over 90% by caching the same queries in the same chat session. |
| **Visualisation** | React Flow | Provides an interactive graph visualisation library to assist with code understanding. |
| **Ingestion** | Octokit (GitHub) | A library for secure fetching of code repositories from GitHub. |
| **Analytics** | Sentry | Provides real-time error and log monitoring, and performance tracking in different environments. |

Table 3.1: *Summary of Technical Stack and Rationale*

# Chapter 4

# Implementation Details

## 4.1 Development Methodology

### 4.1.1 Iterative Development Process

CodeOrient was developed using an iterative approach to continuously refine and integrate feedback. The key stages of the development process are outlined below:

1. **Requirement Analysis:** Pain points for new developers navigating unfamiliar codebases were identified to gather initial requirements.
2. **Prototyping:** To validate core concepts, early prototypes of the search and Generative UI systems were built. For example, a simple weather card UI was created to test the Generative UI's capabilities, as shown in *Figure 4.1*.

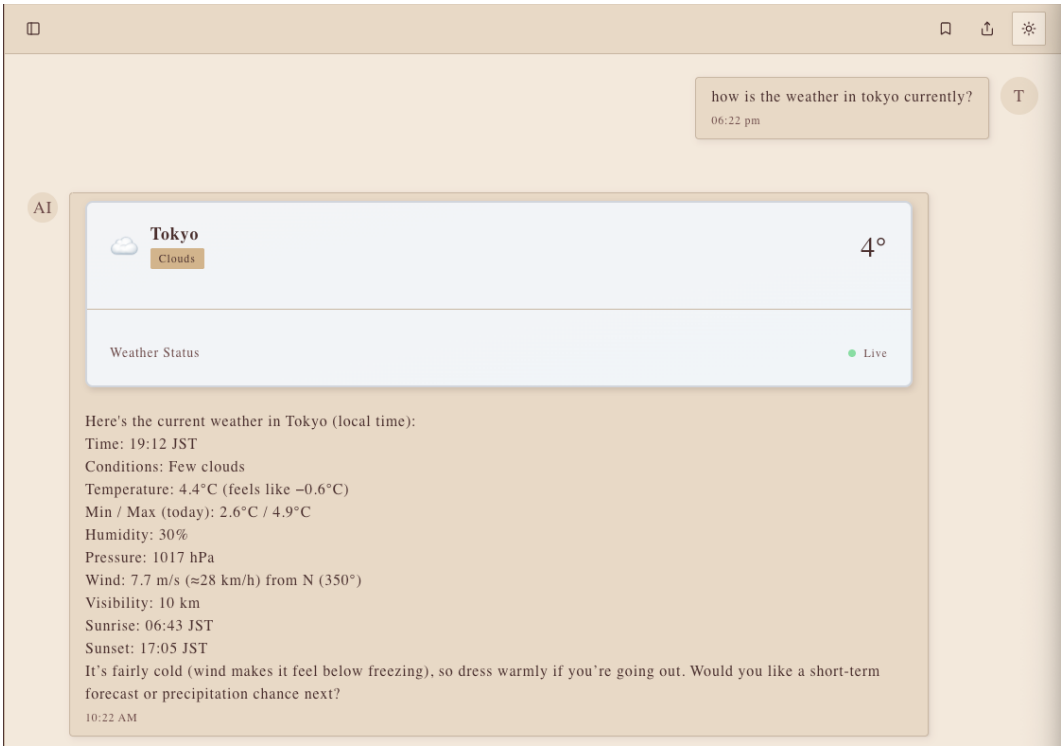



Figure 4.1: *Prototype of Dynamic Card Generation for Weather App*

3. **Incremental Development:** The search module, code graph analysis, and RAG pipeline were identified as priority features. Development of these features was done incrementally to gather early feedback.

4. **Integrating User Feedback:** User testing feedback was regularly incorporated to improve the CodeOrient user experience.

5. **Final Testing and Optimisation:** The system underwent rigorous testing and automated deployment via CI/CD pipelines to ensure the performance and reliability of production release.

### 4.1.2 Version Control and Branching Strategy

This project utilised Git and GitHub for version control. Feature-branching was used to isolate the development of core features. This ensured that the main branch remained stable and protected for user testing. As for deployment, it was automated through CI/CD pipelines and was deployed to Vercel. *Figure 4.2* illustrates the branching strategy which, squashes feature branches into the main branch after code reviews and

testing.

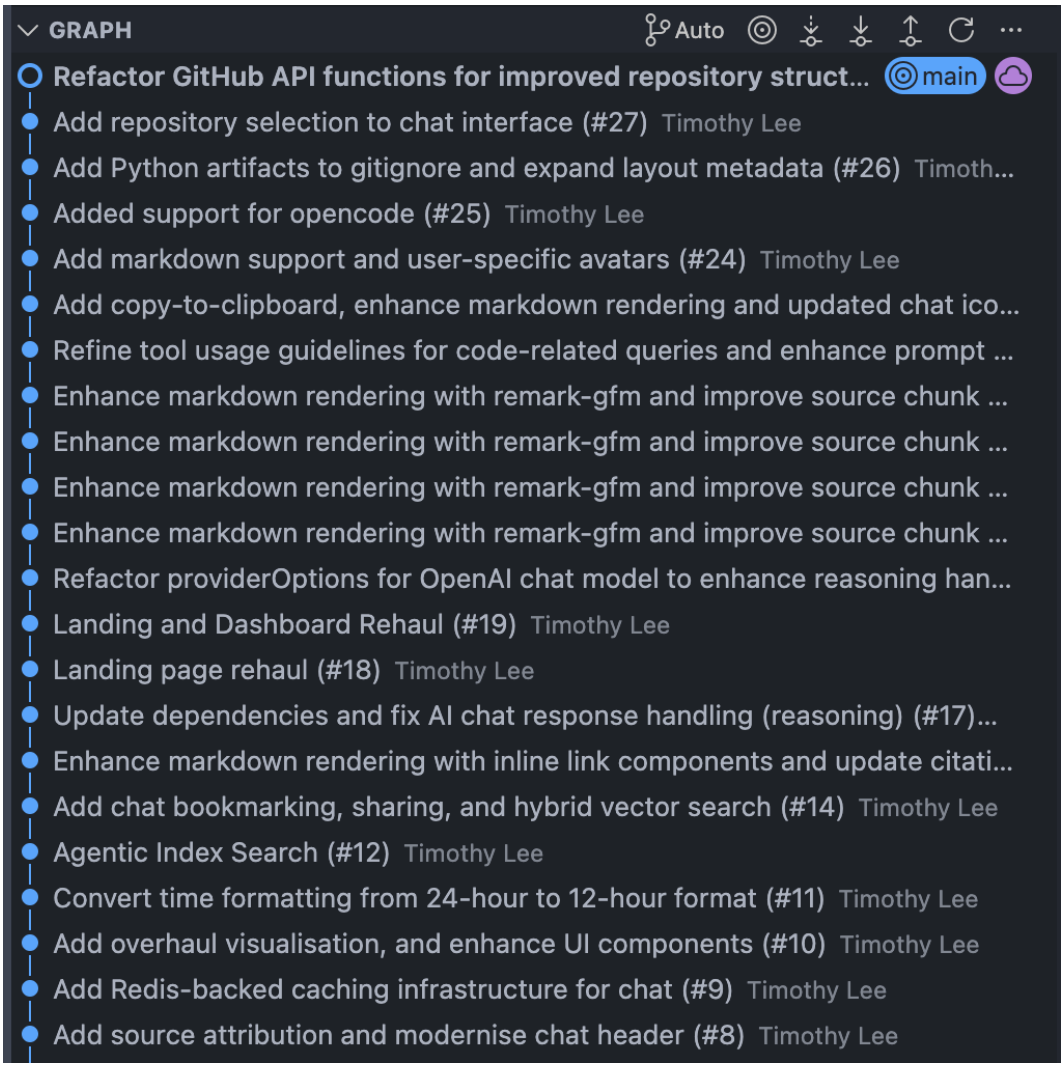



Figure 4.2: *Git Branching Strategy for CodeOrient Development*

## 4.2 Core Implementation Components

### 4.2.1 Search Module

#### BM25 Sparse Retrieval

To handle keyword-based searches, the system implements the Best Matching 25 (BM25) algorithm. The BM25 score for a document $D$ given a query $Q$ is computed

as:

$$\text{BM25}(D, Q) = \sum_{q_i \in Q} IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

where:

- $f(q_i, D)$ is the term frequency of query term $q_i$ in document $D$.
- $|D|$ is the length of document $D$.
- $avgdl$ is the average document length in the corpus.
- $k_1$ and $b$ are hyperparameters, typically set to $k_1 = 1.5$ and $b = 0.75$ for general text.
- $IDF(q_i)$ is the inverse document frequency of term $q_i$, which is calculated as:

$$IDF(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$$

  where $N$ is the total number of documents and $n(q_i)$ is the number of documents containing term $q_i$.

CodeOrient uses this retrieval method to find code snippets that match the keywords in the user query. For example, if a user searches for an exact function name, BM25 will prioritise documents that contain that term.

#### Dense Embedding Retrieval

`bge-large-en-v1.5` embedding model is used for dense retrieval. Each chunk of code is converted to a 1024-dimensional vector and stored in Upstash Vector. This allows the engine to retrieve code snippets based on semantic similarity to the user query. The similarity between the query vector $Q$ and document vector $D$ is computed using cosine similarity:

$$\text{cosine_similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\|\|\vec{b}\|}$$

#### Hybrid Search Integration

The final ranking is achieved through Distributed-Based Score Fusion. The normalised score is computed as:

$$Score = \frac{s - (\mu - 3\sigma)}{(\mu + 3\sigma) - (\mu - 3\sigma)}$$

where:

- $s$ is the score.
- $\mu$ is the mean of the scores.
- $\sigma$ is the standard deviation.
- $(\mu - 3\sigma)$ represents the minimum value (lower tail of the distribution).
- $(\mu + 3\sigma)$ represents the maximum value (upper tail of the distribution).

This approach takes into account the distribution of scores, which is more sensitive to variation in score ranges from the different retrieval methods.

### 4.2.2 Code Graph Analysis

This section details the transformation of the code chunks into an interactive graph.

#### Dependency Extraction

Rather than using AST for traversal, the system utilises a recursive language-aware splitting strategy. This approach splits the code into smaller chunks while maintaining the programming language's nuances, syntax and structure. This strategy is outlined in *Algorithm 1*.

**Algorithm 1:** *Language-Aware Recursive Code Splitting Algorithm*

**Input:** File Content $C$, File Extension $E$, ChunkSize $S$, Overlap $O$

**Output:** List of Semantic Chunks $K$

$K \leftarrow \emptyset$

$Language \leftarrow$ MapExtensionToLanguage($E$)

**if** *Language is supported* **then**

    $Splitter \leftarrow$ InitialiseLangChainSplitter($Language, S, O$)

**end**

**else**

    $Separators \leftarrow$ {"\nclass ", "\ndef ", "\n\n", "\n", " "}

    $Splitter \leftarrow$ InitialiseRecursiveSplitter($Separators, S, O$)

**end**

$Documents \leftarrow Splitter$.splitText($C$)

**foreach** $Doc$ *in* $Documents$ **do**

    $Chunk \leftarrow$ ExtractContentAndMetadata($Doc$)

    Add $Chunk$ to $K$

**end**

**return** $K$

### Graph Construction Algorithm

As React Flow requires a structured object to render the graph, the extracted entities and their relationships are mapped to a JSON object. In *Listing 1*, the code entities are represented as nodes and their relationships as edges.

```typescript
/**
 * Schema for Code Graph Nodes
 */
export type CodeGraphNode = {
  id: string; // Unique identifier: userId::repo::path::type::name
  label: string; // The display name of the entity
  type?: "file" | "function" | "class" | "component";
  filePath?: string; // Original source file path
  codeSnippet?: string; // The raw source code associated with the entity
  description?: string; // Extracted docstring or JSDoc comment
};

/**
 * Schema for Code Graph Edges
 */
export type CodeGraphEdge = {
  id: string; // Composite ID: sourceID->targetID
  source: string; // ID of the originating node
  target: string; // ID of the destination node
  label?: string; // Relationship type
  type?: "imports" | "calls" | "extends" | "uses";
  animated?: boolean; // Visual indicator for logic flow
};
```

Listing 1: *TypeScript interfaces for CodeOrient graph entities.*

### 4.2.3 Generative UI System

The Generative UI system dynamically creates interactive UI cards based on user queries. The implementation involves several key components:

#### Toolkit Selection

External tools are integrated to enhance the LLM's capabilities. The available tools for selection are:

- **Code Graph Tool:** Retrieves and visualises code entities and their relationships.

- **Repository Search Tool:** Fetches all repositories associated with the user.
- **GitHub Search Tool:** Fetches specific files or code snippets from GitHub directly.
- **Vector Search Tool:** Interacts with the hybrid search module to fetch relevant code snippets.

### Dynamic Card Generation

Based on the user query, the LLM intelligently selects the appropriate tool(s) to fulfil the request. It then generates a structured JSON object based on the chosen tool(s), resulting in different card visualisations. It is dynamically streamed to the frontend for real-time rendering. The different visualisations supported are:

- **Repository Card**

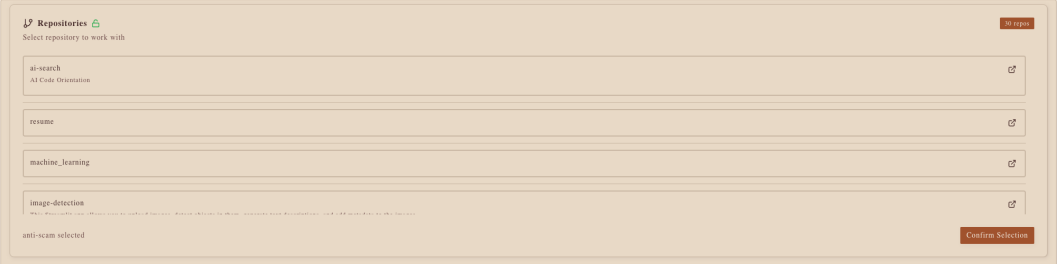



Figure 4.3: *Example of Repository Card in Generative UI*

- **Code Graph Card**

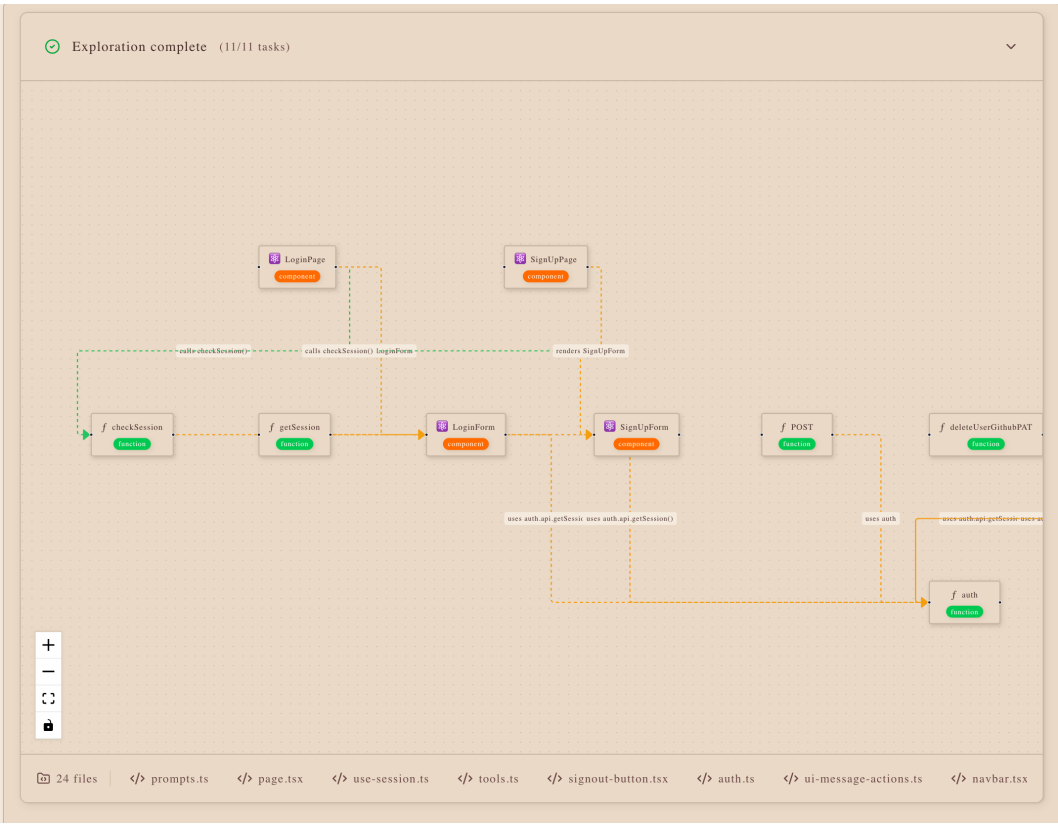



Figure 4.4: *Example of Code Graph Card in Generative UI*

### 4.2.4 Large Language Model Integration

CodeOrient is model-agnostic and can integrate with any LLM providers that support tool-calling.

#### Model Selection and Justification

In this implementation, CodeOrient utilises OpenAI's GPT-4.1-mini model due to its advanced reasoning capabilities and large context window, which prevents "lost in the middle" degradation. Furthermore, the latency and cost-effectiveness of the mini variant make it suitable for real-time applications compared to State-Of-The-Art reasoning models.

#### Prompt Engineering Strategies

To optimise the performance of the LLM and reduce hallucinations during code exploration, a Multi-layered Prompting Strategy is employed with specialised personas for

different stages of the RAG pipeline:

- **Search Architect Persona:** This persona's responsibility is to decompose a user's query into a structured search plan. The plan entails the steps it will take before generating a graph. The most important responsibility is to break down complex queries into non-overlapping sub-queries that target different parts of the codebase. Additionally, the repository's tree is provided as context to guide the planning process. This reduces the hallucination of non-existent files or functions. In *Figure 4.5*, the step-by-step breakdown of the planning process is illustrated to the user.

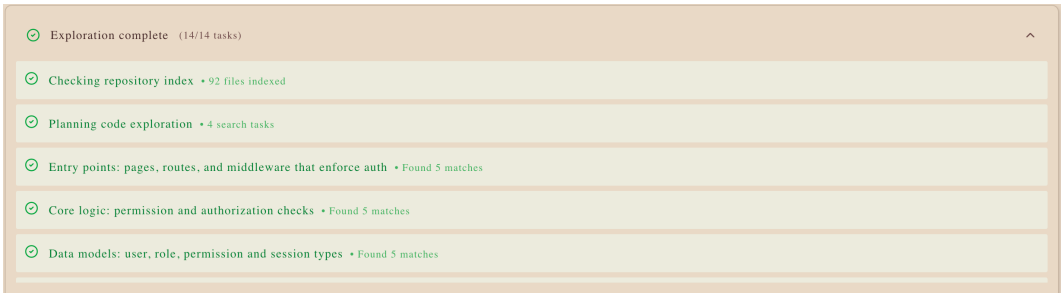



Figure 4.5: *Example of Multistep Planning by Search Architect Persona*

- **Gap Analyser Persona:** To give the LLM autonomy in exploring the codebase, this persona acts as a quality control layer to identify gaps in the retrieved context. An additional search iteration with refined queries will be triggered if the context is deemed insufficient. This iterative process continues until the LLM can accurately answer the user's query. *Figure 4.6* illustrates an example of the LLM identifying a need for additional context.

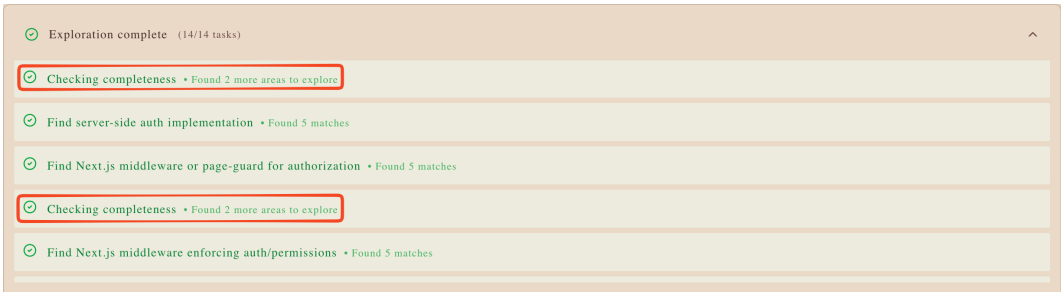



Figure 4.6: *Example of Gap Analysis by Gap Analyser Persona*

- **Graph Architect Persona:** As React Flow requires a structured object to render

the graph, this persona translates the retrieved context into a graph object. This is achieved by identifying the relevant entities and their relationships, before formatting them into nodes and edges.

### 4.2.5 RAG Pipeline Implementation

This pipeline iteratively retrieves relevant code snippets to ground the LLM's responses. The key components are outlined below:

#### Query Processing

User queries are usually ambiguous and may include typos. To address this, the pipeline first rephrases the query into technical search queries (using the Search Architect persona) that align with the codebase's terminology and structure.

#### Multistep Planning & Exploration

Mentioned previously, the Search Architect persona decomposes complex queries and conducts a breadth-first exploration.

#### Retrieval & Ranking

The refined queries are used to fetch relevant code chunks from the hybrid search module, with `userId` and `repoFullName` used as filters to ensure strict multi-tenancy. K-Nearest Neighbour with $K = 10$ is used to retrieve the most similar chunks based on cosine similarity. The retrieved chunks are then ranked using the Distributed-Based Score Fusion method to ensure the most relevant snippets are prioritised.

#### Context Assembly

The top-ranked code chunks are assembled and wrapped in XML-style tags containing metadata for the LLM to accurately reference the sources during the response phase. An example of the assembled context is shown in *Listing 2*.

```
<chunk file="lib/auth.ts" lines="12-45" url="...">
  [Code Snippet]
</chunk>
```

Listing 2: *Example of Assembled Context with Source Metadata.*

### Citation Extraction and Grounding

To increase the credibility of the generated answer, all sources retrieved from the RAG pipeline are shown to the user in *Figure 4.7*. To further reduce hallucination in the response, the LLM is prompted to provide inline citations using markdown format `[file_path](link_to_source_code)`. The frontend parses these citations to create clickable links that direct users to the exact source code locations. An example of inline citations is shown in *Figure 4.8*.

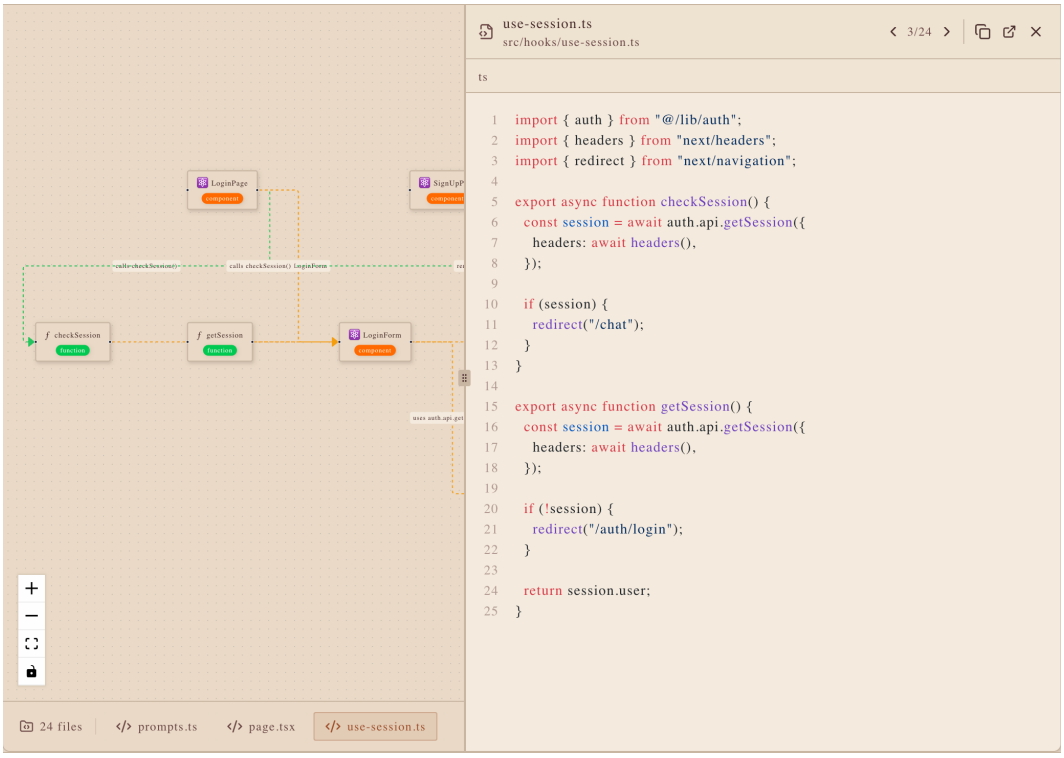
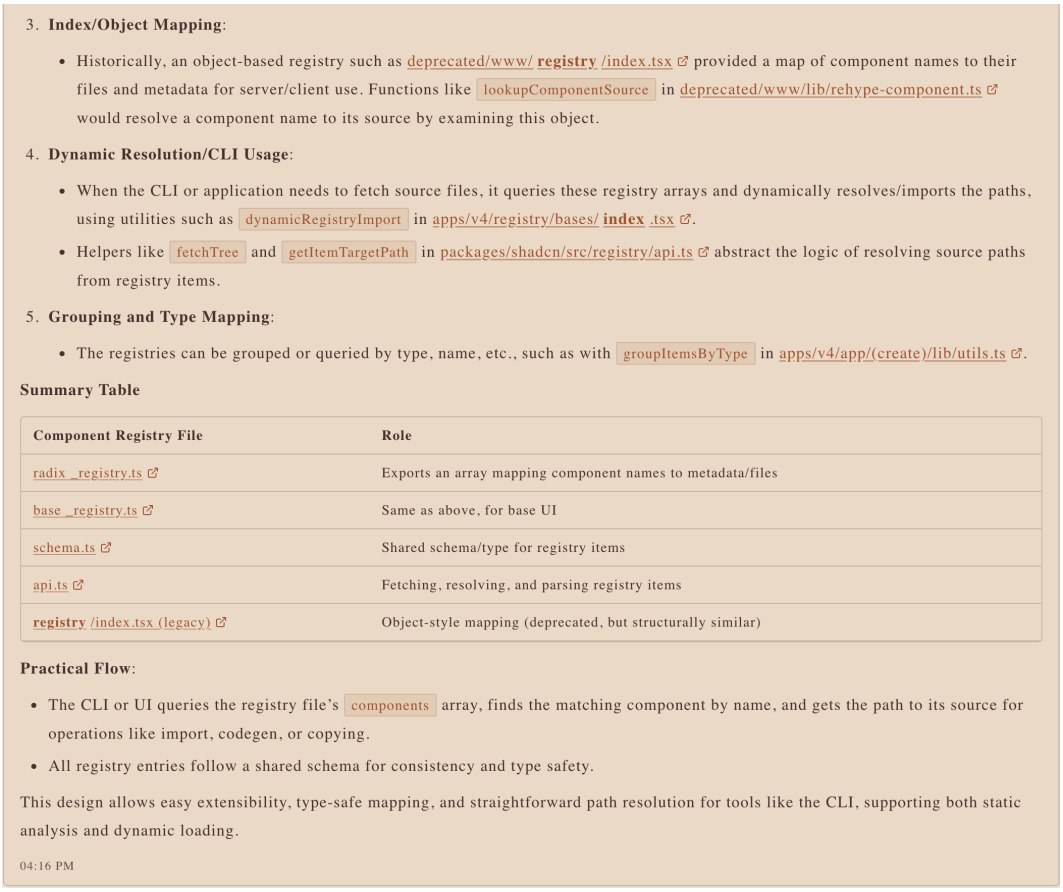



Figure 4.7: *Example of sources retrieved from the RAG pipeline*



3. **Index/Object Mapping**:
   - Historically, an object-based registry such as deprecated/www/ **registry** /index.tsx provided a map of component names to their files and metadata for server/client use. Functions like `lookupComponentSource` in deprecated/www/lib/rehype-component.ts would resolve a component name to its source by examining this object.
4. **Dynamic Resolution/CLI Usage**:
   - When the CLI or application needs to fetch source files, it queries these registry arrays and dynamically resolves/imports the paths, using utilities such as `dynamicRegistryImport` in apps/v4/registry/bases/ **index** .tsx.
   - Helpers like `fetchTree` and `getItemTargetPath` in packages/shadcn/src/registry/api.ts abstract the logic of resolving source paths from registry items.
5. **Grouping and Type Mapping**:
   - The registries can be grouped or queried by type, name, etc., such as with `groupItemsByType` in apps/v4/app/(create)/lib/utils.ts.

**Summary Table**

| Component Registry File | Role |
|---|---|
| radix _registry.ts | Exports an array mapping component names to metadata/files |
| base _registry.ts | Same as above, for base UI |
| schema.ts | Shared schema/type for registry items |
| api.ts | Fetching, resolving, and parsing registry items |
| **registry** /index.tsx (legacy) | Object-style mapping (deprecated, but structurally similar) |

**Practical Flow**:

- The CLI or UI queries the registry file's `components` array, finds the matching component by name, and gets the path to its source for operations like import, codegen, or copying.
- All registry entries follow a shared schema for consistency and type safety.

This design allows easy extensibility, type-safe mapping, and straightforward path resolution for tools like the CLI, supporting both static analysis and dynamic loading.

04:16 PM

Figure 4.8: *Example of inline citations in LLM response*

## 4.3 Database Schema

CodeOrient utilises PostgreSQL for relational data storage, managed via Prisma ORM. The schema is organised into three primary clusters:

- **User Identity & Session:** Tables to manage user authentication and GitHub Personal Access Tokens (PATs).
- **Conversation State:** Tables storing `Chat`, `Message`, and `Part` models to support Generative UI and tool-calling outputs.
- **Indexing Lifecycle:** Table to track the asynchronous indexing jobs for user repositories, used in the RAG pipeline.

The complete Prisma schema is provided in *Listing 3*.

```
// Core User and Repository Models
model User {
  id            String    @id
  name          String
  email         String    @unique
  githubPAT     String?   // Encrypted token for repository access
  chats         Chat[]
  createdAt     DateTime  @default(now())
  @@map("user")
}

model IndexedRepository {
  id            String         @id @default(cuid())
  userId        String
  repoFullName  String         // Format: "owner/repo"
  status        IndexingStatus @default(PENDING)
  progress      Int            @default(0)
  totalFiles    Int            @default(0)
  indexedFiles  Int            @default(0)
  lastIndexedAt DateTime?

  @@unique([userId, repoFullName])
  @@map("indexed_repository")
}

// Conversational State + Generative UI Support
model Chat {
  id        String    @id @default(cuid())
  title     String
  messages  Message[]
  userId    String?
  User      User?     @relation(fields: [userId], references: [id])
  @@map("chat")
}

model Message {
  id        String      @id @default(cuid())
```

```
38    chatId    String
39    chat      Chat        @relation(fields: [chatId], references: [id],
      ↪  onDelete: Cascade)
40    role      MessageRole
41    parts     Part[]      // Supports multi-modal and tool-call outputs
42    @@map("message")
43  }
44
45  model Part {
46    id        String          @id @default(cuid())
47    messageId String
48    message   Message         @relation(fields: [messageId], references: [id],
      ↪  onDelete: Cascade)
49    type      MessagePartType
50
51    // Generative UI and Tool Metadata
52    tool_toolCallId           String?
53    tool_visualiseCodeGraph_output Json?
54    data_codeGraph            Json?
55
56    @@map("part")
57  }
58
59  enum IndexingStatus {
60    PENDING
61    CLONING
62    PARSING
63    INDEXING
64    COMPLETED
65    FAILED
66  }
```

Listing 3: *Prisma Database Schema.*

## 4.4 UI/UX Enhancements

To further improve the user experience in CodeOrient, several additional features were implemented:

### 4.4.1 Account Dashboard

The account dashboard allows users to monitor any activity related to their account. As shown in *Figure 4.9*, users can view statistics such as:

- Recent Activities
- Usage statistics (e.g., number of lifetime searches, total repositories analysed over time)

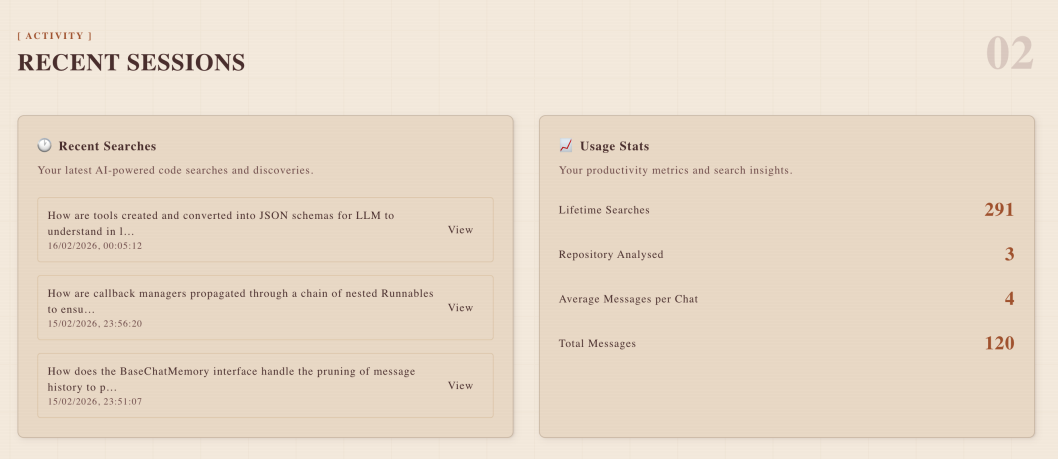



Figure 4.9: *Example of Account Dashboard*

### 4.4.2 Sharing of Conversations

As collaboration is a key aspect of software development, CodeOrient supports the sharing of conversations via unique read-only links. *Figure 4.10* demonstrates the shared chat view where authorised users can view the conversation history in read-only mode.




Figure 4.10: *Example of Sharing Conversation Link*

### 4.4.3 Setting Preferences

This interface allows users to customise their experience. As illustrated in *Figure 4.11*, this tab of the settings page focuses on data integration. Users can securely connect their GitHub accounts via Personal Access Tokens (PATs) to enable repository indexing and analysis. Furthermore, users can index any public or private repositories they have access to. The entire indexing process is asynchronous, and real-time progress updates are provided. By enabling indexing, the LLM will prioritise searching the vector database over GitHub to reduce latency and cost.

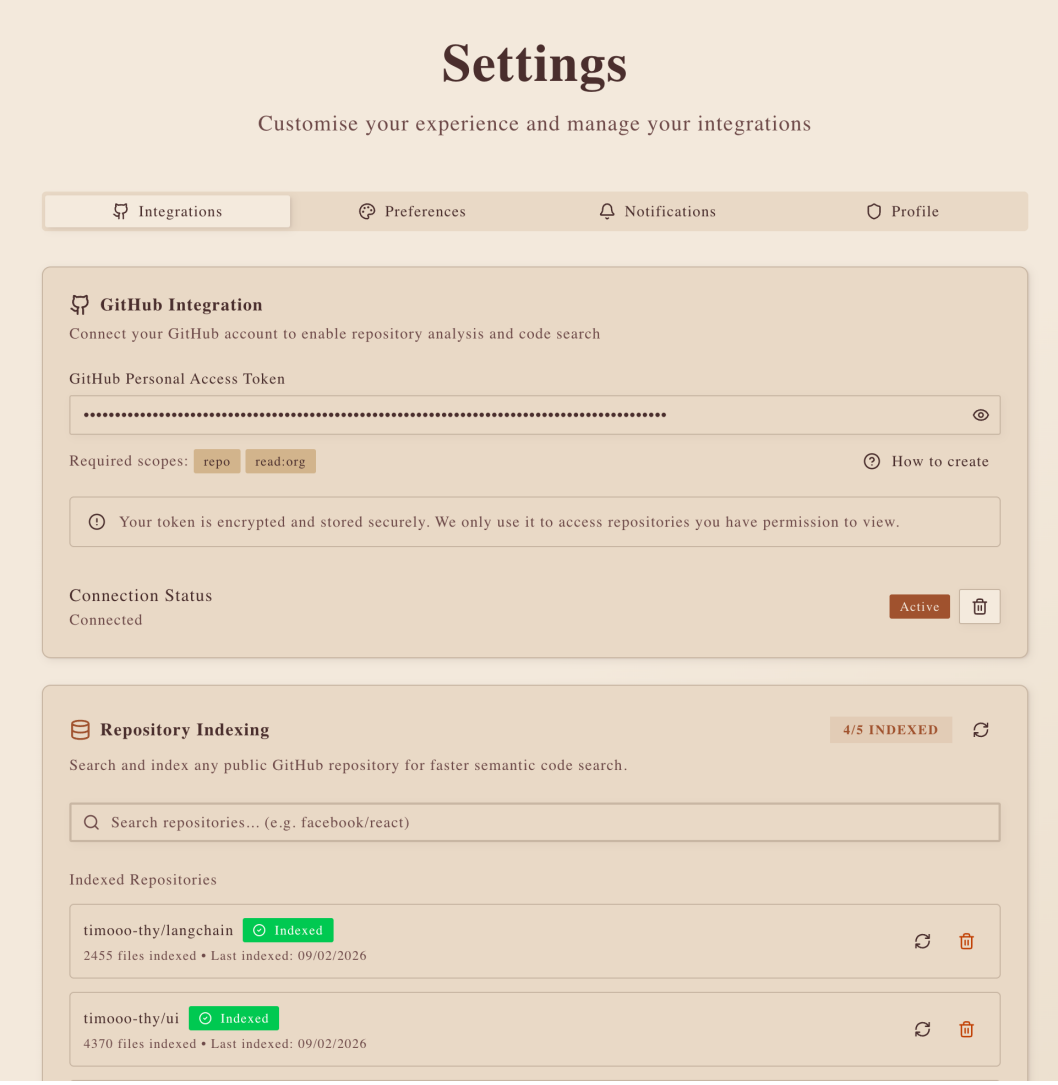



Figure 4.11: *Example of User Preferences Settings*

## 4.5 Challenges and Solutions

### 4.5.1 Handling Large Codebases

As GitHub API has strict rate limits, retrieving large codebases can be time-consuming. To address this challenge, the system implements a **Streaming Tarball Strategy**. Instead of making multiple API calls, the system fetches a single compressed `.tar.gz` archive, reducing network calls by over 90% and processing repository data entirely in-memory.

### 4.5.2 Real-Time Graph Updates

LLMs can take several seconds to generate responses, leading to a suboptimal user experience. Hence, Server-Sent Events (SSE) is used to stream the graph data incrementally, which allows React Flow to render an empty canvas initially and progressively add nodes and edges after the generation has completed.

# Chapter 5

# Evaluation Methodology

This chapter outlines the methodology used to evaluate the performance, accuracy and efficiency of CodeOrient. Quantitative metrics are recorded to conduct comparative analysis against baseline methods.

## 5.1 Evaluation Framework

The primary goal of the evaluation is to assess how well CodeOrient helps developers understand and navigate complex codebases, compared to traditional search methods and ablation models, using retrieval, faithfulness, and graph recall metrics.

### 5.1.1 Research Questions

To guide the evaluation, the following research questions are formulated:

1. **RQ1 (Retrieval Accuracy):** How does CodeOrient's retrieval performance compare to a baseline GitHub search and an ablation model without RAG?
2. **RQ2 (Generation Quality):** How does CodeOrient mitigate hallucination and ensure the faithfulness of generated explanations and code snippets?
3. **RQ3 (Visual Accuracy):** How accurately does CodeOrient's graph visualisation represent the relevant code entities and their relationships?

4. **RQ4 (Efficiency and User Experience):** How easy and efficient is it for developers to use CodeOrient compared to manual code navigation?

### 5.1.2 Hypotheses

Based on the research questions, the following hypotheses are proposed:

1. **H1:** CodeOrient will achieve a significantly higher recall rate and factual faithfulness than the GitHub search baseline.
2. **H2:** CodeOrient will result in significantly higher recall rates, factual faithfulness, graph recall, and overall latency than the ablation model.
3. **H3:** CodeOrient will drastically reduce manual user effort (measured in Clicks-to-Destination) compared to baseline manual navigation.

## 5.2 Experimental Setup

To rigorously test the hypotheses, an evaluation pipeline is created to simulate real-world developer queries and measure the performance of CodeOrient against baselines.

### 5.2.1 Dataset

The evaluation dataset consists of 30 distinct natural language queries across 3 open-source projects with varying codebase sizes (`fastapi/fastapi`, `shadcn-ui/ui`, `langchain-ai/langchain`). The queries are designed to mimic real-world developer intents. This ranges from locating simple components (e.g., "Where is the accessible dialog overlay logic?") to more complex architectural questions (e.g., "How does the system resolve and inject dependencies?"). Each query was executed across all evaluated methods to establish a direct comparison. The full list of queries is provided in Table A.1.

### 5.2.2 Baseline Comparisons

CodeOrient's performance is compared against two baselines:

- **GitHub Search Baseline:** This baseline uses GitHub's native code search functionality to retrieve relevant code snippets based on the same natural language queries. This represents the traditional manual search that developers usually perform.
- **Ablation Model (GitHub + Agent):** This model uses the same search mechanism as CodeOrient but replaces the RAG pipeline with GitHub's Search API directly. This allows us to evaluate the RAG pipeline's contribution to CodeOrient's overall performance.

## 5.3 Quantitative Metrics

### 5.3.1 Recall@System

This metric measures the recall rate of the retrieved code snippets, which is the proportion of relevant code snippets that were successfully retrieved by CodeOrient compared to the total number of relevant snippets based on ground truths.

### 5.3.2 Success@Turn

This metric measures the average number of turns CodeOrient takes to retrieve the correct code files for a given query. A turn is defined as a search-and-retrieval cycle, with a maximum of 3 turns per query. A lower Success@Turn indicates a more efficient retrieval process. If the system fails to retrieve the correct code files within the threshold, a penalty score of $Max + 1$ (4 in this instance) is assigned to reflect the retrieval failure.

### 5.3.3 Graph Recall

Graph Recall measures the completeness of the generated graph visualisations. It is a measure of how many relevant nodes are successfully rendered in the graph compared to the total number of relevant nodes identified in the ground truth.

### 5.3.4 Faithfulness (Hallucination Rate)

Faithfulness measures the factual alignment of the generated explanations and code snippets with the actual codebase. Having a lower hallucination rate indicates that the system generates responses that are grounded in the retrieved code snippets rather than its pre-trained knowledge.

### 5.3.5 Latency and Caching Efficiency

Latency is a critical metric for evaluating CodeOrient's responsiveness. It is measured in seconds and captures the time taken for the system to process a query and generate a response. Furthermore, the efficiency of caching is evaluated by measuring the latency of repeated queries in the same session that should benefit from cached results.

### 5.3.6 Clicks to Destination (CTD)

CTD is a user-centric metric that evaluates the efficiency of CodeOrient's search and navigation experience. It measures the number of interactions (clicks) required for a user to navigate from the initial query to the relevant code snippets based on ground truths. A lower CTD indicates a more efficient search and navigation experience, as it reflects fewer steps needed for users to reach their desired information.

# Chapter 6

# Results and Discussions

## 6.1 Quantitative Results

The CodeOrient evaluation was conducted across 30 queries spanning three modern, well-maintained repositories. The metrics and results across CodeOrient, the ablation model, and the GitHub Search baseline are summarised in *Table 6.1*.

| Metric | CodeOrient (Full RAG) | GitHub + Agent (Ablation) | GitHub Search (Baseline) |
|---|---|---|---|
| Recall@System | 92.50% | 66.39% | 2.78% |
| Success@Turn | 1.50 | 2.31 | - |
| Latency | 62.79s | 77.83s | - |
| Latency (Cache) | 4.86s | 5.03s | - |
| Faithfulness | 98.43% | 98.33% | - |
| Graph Recall | 93.61% | 69.72% | - |
| CTD | 0.07 | 0.03 | 5.27 |

Table 6.1: *Average Evaluation Metrics Across 30 Queries*

## 6.2 Comparative Analysis with Baselines

### 6.2.1 Performance vs GitHub Search

Comparing CodeOrient to the GitHub Search baseline, there is a significant improvement in both Recall@System (92.50% vs 2.78%) and Clicks to Discovery (0.07 vs 5.27). This showcases the limitations of traditional keyword-based search in understanding natural language queries. Furthermore, the baseline's low recall rate and high

CTD indicate that developers would need to manually sift through search results, which is inefficient and time-consuming. On the contrary, CodeOrient retrieved the required code snippets with a high recall rate and minimal user interaction, demonstrating its effectiveness in enhancing developer productivity.

### 6.2.2 Performance vs Ablation Model

Comparing CodeOrient to the ablation model, which uses the same search mechanism but swaps the RAG pipeline with GitHub's Search API directly, there is a significant improvement in Recall@System (92.50% vs 66.39%), Success@Turn (1.50 vs 2.31), Graph Recall (93.61% vs 69.72%) and Latency (62.79s vs 77.83s).

Firstly, it highlights the importance of the RAG pipeline in effectively retrieving relevant code snippets and generating accurate graph visualisations. Secondly, the improvement in Success@Turn indicates that CodeOrient is more efficient at retrieving the correct code files in fewer turns, which directly ties to the improved latency.

Thirdly, the similar faithfulness scores between the two models suggest that the multi-layered prompting strategy in CodeOrient successfully mitigates most hallucination issues, and the main performance gains are attributed to the enhanced retrieval and graph generation capabilities provided by the RAG pipeline. Lastly, the use of caching in CodeOrient reduced the latency of repeated queries by an average of 92.2% and 93.5% for CodeOrient and the ablation model, respectively, which demonstrates the efficiency of caching in improving response times for repeated queries.

## 6.3 Key Findings

### 6.3.1 Impact of Graph Visualisation and Emergent Inference

With a Recall@System of 92.50%, CodeOrient successfully bridges the gap between code retrieval and architectural understanding by generating accurate graph visualisations that capture the relationships between code snippets.

Instances of "Emergent Inference" were observed, where the system identified imported

relationships that were not explicitly retrieved, allowing the system to maintain an accurate graph visualisation even when some relevant files were missed in the initial retrieval step. This is evident in the average Graph Recall (93.61%) being higher than the average Recall@System (92.50%), suggesting that the system inferred relationships between code snippets that were not directly retrieved.

### 6.3.2 Effectiveness of Citation Grounding

The high Faithfulness score of 98.43% is directly attributable to the citation grounding mechanism. Prompting the LLM to provide inline citations when generating responses ensures that the information is directly traceable to specific code snippets, significantly reducing the likelihood of hallucination.

## 6.4 Case Studies

Across the 30 queries, there were several interesting cases that highlighted the strengths and limitations of CodeOrient.

### 6.4.1 Case Study 1: "How does the system resolve and inject dependencies?"

CodeOrient achieved only a Recall@System of 50% for this complex architectural query. While CodeOrient located the primary dependency utility file (`utils.py`), it failed to retrieve the underlying implementation.

This highlights a classic "multi-hop" challenge in code retrieval, where the recursive nature of dependencies spans multiple files. This suggests a gap in the system's ability to effectively traverse deeper levels of dependencies, which is crucial for queries that require understanding the full execution flow of a codebase.

This directly impacts graph recall performance, with the system achieving 50% recall for this query. This indicates that while the primary nodes were retrieved, some of the secondary nodes (e.g., files containing the underlying implementation) were missed. This case study highlights an area for future improvement in CodeOrient, which is

to enhance the multi-hop retrieval capabilities to ensure that all relevant files across multiple levels of dependencies are retrieved effectively.

### 6.4.2 Case Study 2: "How are the button visual variants defined?"

This query demonstrates a scenario in which, even though the system achieved a Recall@System of 50%, the generated graph visualisation achieved a graph recall of 100%. This showcases the emergent inference capabilities of the LLM when it comes to understanding import relationships in the code snippets.

In this case, CodeOrient successfully retrieved the main file defining the button variants (`button.tsx`) and, through citation grounding and multi-layered prompting, inferred the relevant related file that was not directly retrieved (`utils.ts`). This is shown in *Figure 6.1*, where the inferred node is highlighted in red.

This highlights a unique strength of CodeOrient, which is its ability to leverage the reasoning capabilities of LLMs to fill in gaps in retrieval and generate more complete graph visualisations, even when some relevant files are missed in the initial retrieval step. This also suggests that while improving recall is important, enhancing the system's reasoning and inference capabilities can significantly improve graph recall and overall user experience.

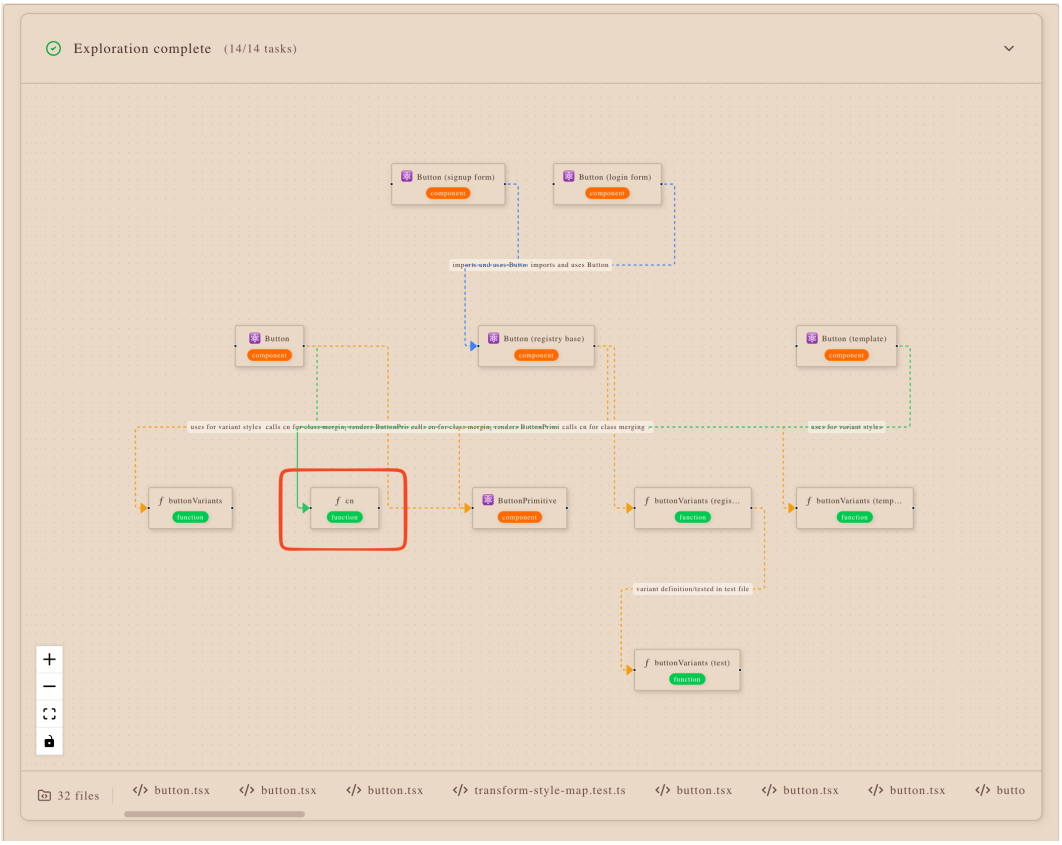



Figure 6.1: *Graph visualisation for Case Study 2*

### 6.4.3 Case Study 3: "How is dark mode theme switching implemented?"

This query serves as a justification for the use of the LLM Evaluation Loop in the system. This query required identifying three specific files: `active-theme.ts`, `theme-selector.tsx` and `mode-switcher.tsx`.

CodeOrient achieved a perfect Recall@System of 100% for this query with a Success@Turn of 3-2-1. This meant the system refined its search over three turns, resulting in the successful retrieval of all three files. In contrast, the ablation model stalled at a Recall@System of 67%.

Hence, this example highlights the system's ability to identify gaps in the initial retrieval results and refine its search queries in subsequent turns. This resulted in a complete graph visualisation as shown in *Figure 6.2*, which accurately captured the relationships between the three files.

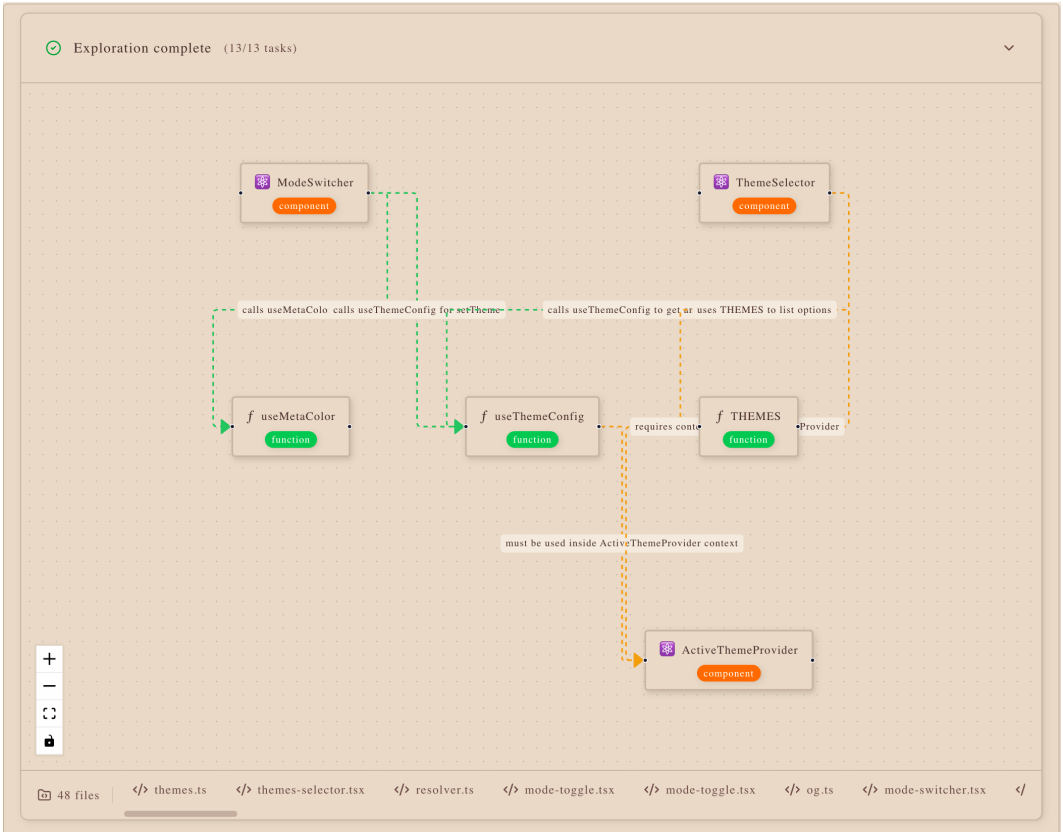



Figure 6.2: *Graph visualisation for Case Study 3*

## 6.5 Implications for Developer Onboarding

The results of this evaluation showcased a reduction of CTD (Clicks to Discovery) from 5.27 with the GitHub Search baseline to 0.07 with CodeOrient. This significant reduction in CTD demonstrates how CodeOrient effectively eliminates the need for developers to manually sift through search results, which is a common pain point in traditional code search. A lower CTD suggests that developers can spend more time understanding and working with the code rather than searching for it, which can significantly enhance productivity and reduce the time taken for onboarding.

In addition, the high graph recall and faithfulness scores demonstrate that CodeOrient can provide accurate graph visualisations to help new developers understand the relationships among codebase components. This can be particularly beneficial for onboarding, as it allows new developers to quickly grasp the overall structure of the codebase and how its components interact.

## 6.6 Limitations

### 6.6.1 Technical Limitations

#### Multi-Hop Retrieval Challenge

The "multi-hop" retrieval mentioned in subsection 6.4.1 showcases how the system struggles with "multi-hop" dependency retrieval. As the system currently limits the evaluation loop to 3 turns, it may not be able to retrieve all relevant files. This is a common trade-off between latency and recall, as allowing for more turns could improve recall but would also increase latency.

#### Latency

Without caching, the system's average latency is 62.79s. The RAG pipeline and Evaluation Loop, which requires multiple calls to the LLM and vector database, are the main contributors to this latency. Although caching strategies have been implemented to reduce latency for repeated queries, cold-start latency remains a concern for user experience. One solution implemented was to stream the response as it is generated, reducing the perceived latency for users.

#### Context Window

Although the context window of LLMs has increased significantly, it is still limited compared to the size of codebases. This can pose challenges for queries that require retrieving many relevant files. If the retrieved code snippets exceed the context window, the generated responses may be incomplete or inaccurate.

### 6.6.2 Threats to Validity

#### Internal Validity

As LLMs are non-deterministic, there may be variations in results across evaluation runs. This could potentially affect the internal validity of the evaluation. In order to

migate this issue, a temperature of 0.0 was used for all LLM inference calls to ensure a more deterministic and reproducible output.

Additionally, it is important to acknowledge that evaluating faithfulness and graph recall involves some subjectivity. To address this, clear guidelines and criteria were established for scoring these metrics to ensure consistency and reduce bias.

### External Validity

The evaluation was conducted on a golden set of 30 queries across three modern, well-maintained repositories. The queries were designed to be representative of common developer information needs, but they may not cover the full range of queries developers encounter in real-world scenarios. Additionally, the repositories chosen for evaluation may not be representative of all types of codebases, such as legacy or poorly documented codebases. Therefore, the generalisability of the results to other queries and repositories may be limited.

### Construct Validity

While Clicks to Discovery (CTD) measures the speed of information discovery, it does not necessarily reflect the depth of a developer's understanding. However, the high faithfulness and graph recall scores suggest that the information provided by CodeOrient is not only quickly discoverable but also accurate and comprehensive, thereby contributing to a deeper understanding of the codebase.

# Chapter 7

# Conclusion

## 7.1 Summary of Contributions

This project has successfully developed CodeOrient, an autonomous AI Search tool to accelerate developer onboarding. The key contributions of this work include:

1. The design and implementation of CodeOrient, which integrates semantic code search, graph visualisation and generative user interfaces.
2. The development of a multi-stage retrieval and ranking pipeline that utilises retrieval-augmented generation (RAG) with source grounding to eliminate hallucination inherent in AI code assistants.
3. The creation of generative user interfaces, such as graph visualisation cards that dynamically render parts of a codebase tailored to user queries and context.
4. The development of Search Architect, Gap Analyser and Graph Architect agents to automate multi-turn exploration of codebases and generate context-aware feature cards.

## 7.2 Key Takeaways

The key takeaways from this project include:

1. Quantitative evaluation of CodeOrient's RAG framework on real-world open-source codebases has demonstrated a 92.5% Recall@System, which is a significant improvement over the 2.78% recall rate of traditional keyword-based GitHub search.

2. CodeOrient's platform reduced the Clicks to Discovery metric from 5.27 to 0.07, demonstrating that the AI search tool can help developers find relevant code snippets with a single click.

3. CodeOrient's graph visualisation achieved a 93.61% recall rate in correctly identifying relevant nodes for the user query, outperforming its own recall by inferring imported relationships that were not explicitly retrieved in the initial search results.

4. 98.43% of the claims made by the system were grounded in the source code, effectively eliminating hallucinations and any possible "slopsquatting" or risks associated with AI-generated code.

## 7.3 Future Work

The proof-of-concept implementation of CodeOrient has shown promising results, but there are several areas of improvement and future work to explore:

1. Future iterations could address the multi-hop retrieval problem by focusing on graph traversal techniques to discover code snippets that are deeply connected but not directly retrieved in the initial search results.

2. Further exploration of latency-optimisation techniques, such as caching and parallelisation, is needed to ensure CodeOrient provides faster responses during cold starts and when handling larger codebases.

3. Further work is needed to manage context effectively and avoid performance degradation as context length increases, such as implementing context window management strategies or summarisation techniques.

4. Integrating the platform directly into IDEs to enhance CodeOrient's accessibil-

ity and usability for developers, allowing them to search without leaving their development environment.

# Appendix A

# Detailed Results

## A.1 Quantitative Results Table

| Query & Repository | Method | Recall | S@T | Lat/Cache | F/GR | CTD |
|---|---|---|---|---|---|---|
| **Repository: fastapi/fastapi** | | | | | | |
| 1. How does the system resolve and inject dependencies? | CodeOrient | 0.50 | 1-NA | 43.3/4.7 | 1.0/0.5 | 0 |
| | Ablation | 0.50 | 1-NA | 79.2/7.7 | 1.0/0.5 | 0 |
| | Baseline | 0.00 | - | - | - | 3.5 |
| 2. Where is the logic for generating the OpenAPI JSON schema? | CodeOrient | 1.00 | 1-2-2 | 44.6/3.9 | 1.0/1.0 | 0 |
| | Ablation | 0.33 | 1-NA-NA | 102.7/5.7 | 1.0/0.3 | 0 |
| | Baseline | 0.33 | - | - | - | 2.3 |
| 3. How are CORS headers handled in the middleware? | CodeOrient | 1.00 | 2 | 33.7/2.8 | 0.8/1.0 | 0 |
| | Ablation | 1.00 | 1 | 67.9/4.6 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 4.0 |
| 4. How does FastAPI handle WebSocket handshakes? | CodeOrient | 1.00 | 1-2 | 40.4/4.1 | 1.0/0.5 | 1 |
| | Ablation | 1.00 | 1-1 | 80.0/5.5 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 3.0 |

Continued on next page

**Table A.1 – continued from previous page**

| Query & Repository | Method | Recall | S@T | Lat/Cache | F/GR | CTD |
|---|---|---|---|---|---|---|
| 5. How does a custom JSON encoder work? | CodeOrient | 1.00 | 1 | 50.9/3.3 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1 | 91.4/4.7 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 3.0 |
| 6. How are background tasks executed? | CodeOrient | 1.00 | 1 | 53.5/3.6 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1 | 94.5/4.5 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 3.0 |
| 7. How to handle the extraction of credentials in password-based auth flow? | CodeOrient | 1.00 | 2 | 56.0/4.1 | 1.0/0.0 | 0 |
| | Ablation | 1.00 | 2 | 82.9/5.8 | 1.0/0.0 | 0 |
| | Baseline | 0.00 | – | – | – | 4.0 |
| 8. How does the system validate request parameters? | CodeOrient | 0.33 | 1-NA-NA | 44.8/3.9 | 1.0/0.3 | 0 |
| | Ablation | 0.33 | 2-NA-NA | 87.1/6.5 | 1.0/0.3 | 0 |
| | Baseline | 0.00 | - | - | - | 4.0 |
| 9. Where is the Swagger UI HTML logic? | CodeOrient | 1.00 | 1-1 | 43.1/2.5 | 1.0/1.0 | 0 |
| | Ablation | 0.00 | NA-NA | 33.5/4.1 | 0.5/0.0 | 0 |
| | Baseline | 0.50 | - | - | - | 2.0 |
| 10. How are HTTP exceptions converted to JSON? | CodeOrient | 1.00 | 2-3 | 47.8/3.1 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1-1 | 75.4/6.3 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 3.0 |
| **Repository: shadcn-ui/ui** | | | | | | |
| 11. How are the button visual variants defined? | CodeOrient | 0.50 | 1-NA | 63.8/6.5 | 1.0/1.0 | 0 |
| | Ablation | 0.50 | 1-NA | 61.1/3.5 | 1.0/0.5 | 0 |
| | Baseline | 0.00 | - | - | - | 7.0 |
| 12. How does the Form component handle validation errors? | CodeOrient | 1.00 | 2 | 80.2/4.2 | 1.0/1.0 | 0 |
| | Ablation | 0.00 | NA | 106.9/6.2 | 1.0/0.0 | 0 |
| | Baseline | 0.00 | - | - | - | 7.0 |

Continued on next page

**Table A.1 – continued from previous page**

| Query & Repository | Method | Recall | S@T | Lat/Cache | F/GR | CTD |
|---|---|---|---|---|---|---|
| 13. Where is the accessible dialog overlay logic? | CodeOrient | 1.00 | 1 | 76.0/3.2 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1 | 97.0/4.2 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 8.0 |
| 14. How is the dark mode theme switching implemented? | CodeOrient | 1.00 | 3-2-1 | 50.0/4.2 | 1.0/1.0 | 0 |
| | Ablation | 0.67 | 1-NA-1 | 115.9/7.0 | 1.0/0.7 | 0 |
| | Baseline | 0.00 | - | - | - | 5.0 |
| 15. How can I create custom tables with filtering and pagination? | CodeOrient | 1.00 | 1-1-1-2 | 69.1/5.8 | 1.0/1.0 | 0 |
| | Ablation | 0.25 | 2-NA-NA-NA | 70.4/6.6 | 1.0/0.5 | 0 |
| | Baseline | 0.00 | - | - | - | 9.0 |
| 16. How does the Calendar component manage date selection? | CodeOrient | 1.00 | 1-1 | 87.5/7.3 | 0.8/1.0 | 0 |
| | Ablation | 0.50 | 1-NA | 59.7/3.1 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 17. What is the configuration for the CLI's init command? | CodeOrient | 0.67 | 1-NA-1 | 79.6/3.7 | 1.0/1.0 | 0 |
| | Ablation | 0.67 | 1-NA-1 | 114.5/5.4 | 1.0/0.7 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 18. How are popover positioning styles calculated? | CodeOrient | 1.00 | 1-3 | 99.4/7.4 | 1.0/1.0 | 0 |
| | Ablation | 0.50 | 1-NA | 55.9/2.9 | 1.0/0.5 | 0 |
| | Baseline | 0.00 | - | - | - | 7.0 |
| 19. How does the framework synchronise Tailwind CSS theme configurations? | CodeOrient | 1.00 | 1-1-1-1 | 95.2/5.8 | 1.0/1.0 | 0 |
| | Ablation | 0.50 | NA-NA-1-3 | 67.2/5.7 | 1.0/0.5 | 0 |
| | Baseline | 0.00 | - | - | - | 6.25 |
| 20. How is the component registry structured for CLI mapping? | CodeOrient | 1.00 | 1-1 | 89.7/7.5 | 1.0/1.0 | 0 |
| | Ablation | 0.00 | NA-NA | 89.7/7.5 | 1.0/0.0 | 0 |
| | Baseline | 0.00 | - | - | - | 5.5 |
| **Repository: langchain-ai/langchain** | | | | | | |

Continued on next page

**Table A.1 – continued from previous page**

| Query & Repository | Method | Recall | S@T | Lat/Cache | F/GR | CTD |
|---|---|---|---|---|---|---|
| 21. What is the base class for all LLM implementations? | CodeOrient | 1.00 | 1 | 57.4/3.2 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 2 | 57.9/3.2 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 22. How does the ”RetrievalQA” chain combine docs? | CodeOrient | 1.00 | 1-1 | 55.1/4.6 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1-1 | 72.7/5.1 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 7.0 |
| 23. What is the logic for the RecursiveCharacterTextSplitter? | CodeOrient | 1.00 | 1-1 | 40.5/5.2 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 2-1 | 50.7/4.2 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 5.0 |
| 24. How can I inject variables into chat prompt templates? | CodeOrient | 1.00 | 1-1-1 | 57.7/5.2 | 1.0/1.0 | 0 |
| | Ablation | 0.33 | NA-1-NA | 49.1/4.8 | 1.0/0.3 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 25. What is the core implementation to store vectors for semantic search? | CodeOrient | 1.00 | 1-1 | 56.7/4.0 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 2-1 | 95.1/2.0 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 5.5 |
| 26. How does the framework abstract differences between LLM providers? | CodeOrient | 1.00 | 1-1-1 | 76.6/7.2 | 1.0/1.0 | 0 |
| | Ablation | 0.67 | NA-2-1 | 77.1/7.2 | 1.0/0.7 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 27. How does ’AgentExecutor’ manage the loop? | CodeOrient | 1.00 | 1-1 | 50.7/4.3 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1-1 | 57.3/3.5 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 28. How does BaseChatMemory handle pruning of message history? | CodeOrient | 1.00 | 1-1-2-1 | 61.7/6.3 | 1.0/1.0 | 0 |
| | Ablation | 1.00 | 1-1-1-3 | 101.1/3.2 | 1.0/1.0 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |

Continued on next page

**Table A.1 – continued from previous page**

| Query & Repository | Method | Recall | S@T | Lat/Cache | F/GR | CTD |
|---|---|---|---|---|---|---|
| 29. How are callback managers propagated through nested Runnables? | CodeOrient | 0.75 | 1-1-1-NA | 111.1/8.2 | 1.0/0.8 | 1 |
| | Ablation | 0.50 | NA-1-NA-1 | 73.5/5.6 | 1.0/0.8 | 1 |
| | Baseline | 0.00 | - | - | - | 6.0 |
| 30. How are tools created and converted into JSON schemas? | CodeOrient | 1.00 | 1-1-1 | 67.5/6.4 | 1.0/1.0 | 0 |
| | Ablation | 0.67 | 1-1-NA | 67.3/6.3 | 1.0/0.7 | 0 |
| | Baseline | 0.00 | - | - | - | 6.0 |

Table A.1: *Complete Quantitative Results by Query*

**Legend: Recall**: Recall@System, **S@T**: Success@Turn (e.g., 1-2 means success at turn 2 for file 2, NA means that the file was not found after 3 turns). **Lat/Cache**: Latency / Latency (Cache) in seconds, **F/GR**: Faithfulness / Graph Recall (1.0 = 100%), **CTD**: Clicks to Discovery.

# Bibliography


[1] T.J. McCabe. "A Complexity Measure". In: *IEEE Transactions on Software Engineering* SE-2.4 (1976), pp. 308–320. DOI: 10.1109/TSE.1976.233837.

[2] Dennis Kafura. "Reflections on McCabe's Cyclomatic Complexity". In: *IEEE Transactions on Software Engineering* 51.3 (2025), pp. 700–705. DOI: 10.1109/TSE.2025.3534580.

[3] José Cambronero et al. "When Deep Learning Met Code Search". In: *CoRR* abs/1905.03813 (2019). arXiv: 1905.03813. URL: http://arxiv.org/abs/1905.03813.

[4] Ben Limpanukorn et al. *Structural Code Search using Natural Language Queries*. 2025. arXiv: 2507.02107 [cs.SE]. URL: https://arxiv.org/abs/2507.02107.

[5] Sheffer Tai. *State-of-the-Art Code Retrieval with Efficient Embeddings*. 2025. URL: https://www.qodo.ai/blog/qodo-embed-1-code-embedding-code-retrieval/.

[6] Joseph Spracklen et al. *We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs*. 2025. arXiv: 2406.10279 [cs.SE]. URL: https://arxiv.org/abs/2406.10279.

[7] Jahidul Arafat. *Citation-Grounded Code Comprehension: Preventing LLM Hallucination Through Hybrid Retrieval and Graph-Augmented Context*. 2025. arXiv: 2512.12117 [cs.SE]. URL: https://arxiv.org/abs/2512.12117.

[8] Orlando Ayala and Patrice Bechard. "Reducing hallucination in structured outputs via Retrieval-Augmented Generation". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational*

*Linguistics: Human Language Technologies (Volume 6: Industry Track)*. Association for Computational Linguistics, 2024, pp. 228–238. DOI: 10.18653/v1/2024.naacl-industry.19. URL: http://dx.doi.org/10.18653/v1/2024.naacl-industry.19.

[9] Jit. *How to Use a Dependency Graph to Analyze Dependencies*. 2025. URL: https://www.jit.io/resources/app-security/how-to-use-a-dependency-graph-to-analyze-dependencies.

[10] Nikola Jovanov et al. "A visual approach to project management using react flow". In: Jan. 2025, pp. 306–312. DOI: 10.5937/IIZS25306J.

[11] Y. Leviathan et al. *Generative UI: LLMs are Effective UI Generators*. 2025. URL: https://research.google/blog/generative-ui-a-rich-custom-visual-interactive-user-experience-for-any-prompt/.

[12] Yining Cao, Peiling Jiang, and Haijun Xia. *Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model*. 2025. arXiv: 2503.04084 [cs.HC]. URL: https://arxiv.org/abs/2503.04084.